

Uniwersytet Mikołaja Kopernika w Toruniu

Wydział Matematyki i Informatyki

Szymon Orzechowski

nr albumu: **308190**

Kierunek studiów: Informatyka

Praca magisterska

**Programowanie równoległe w języku Chapel
na przykładzie problemu rozchodzenia się ciepła**

Promotor

dr Marek Nowicki

Toruń, 2026

Spis treści

Wstęp	3
1 Problem i wykorzystane technologie	5
1.1 Równanie ciepła i metoda różnic skończonych	5
1.1.1 Model fizyczny	5
1.1.2 Warunki początkowe i brzegowe	5
1.1.3 Dyskretyzacja: jawna metoda różnic skończonych	6
1.1.4 Stabilność numeryczna i warunek CFL	7
1.2 Zrównoleglenie obliczeń stencilowych	9
1.2.1 Dekompozycja dziedziny i komórki brzegowe	9
1.2.2 Wymiana brzegów jako jedyna komunikacja	9
1.2.3 Intensywność arytmetyczna i ograniczenia wydajności	10
1.2.4 Miary skalowalności	11
1.3 Chapel, PGAS i GASNet	12
1.3.1 Język Chapel i model PGAS	12
1.3.2 Iteratory równoległe i tablice globalne	13
1.3.3 Konstrukcje równoległości i synchronizacja	14
1.3.4 Warstwa zadań qthreads oraz warstwa komunikacji GASNet	15
1.3.5 Kanały komunikacyjne GASNet	16
2 Klaster i implementacja	18
2.1 Architektura klastra i środowisko	18
2.1.1 Sprzęt i topologia	18
2.1.2 Konfiguracja kompilatora Chapel	19
2.1.3 Dwa kanały komunikacyjne GASNet	20
2.1.4 Awaria kanału udp pod obciążeniem incast	20
2.1.5 Uruchamianie i samoopisujące się logi	21
2.1.6 Procedura wdrożenia krok po kroku	22
2.1.7 Uruchamianie eksperymentu	23
2.2 Struktura programu i krok obliczeniowy	24

2.2.1	Dziedzina rozproszona z otoczką	24
2.2.2	Pojedynczy krok obliczeniowy	25
2.2.3	Ewolucja implementacji i operator zamiany buforów	26
2.2.4	Pomiar czasu i raportowanie	27
2.2.5	Parametry konfiguracyjne	29
2.2.6	Raportowanie przebiegów	29
2.3	Buforowanie, zapis i agregacja danych	30
2.3.1	Zapis fragmentów na dysk lokalny	30
2.3.2	Agregacja rozproszonych fragmentów i wizualizacja	31
3	Metodologia badań	34
3.1	Scenariusze i metryki	34
3.1.1	Scenariusze badawcze	34
3.1.2	Metryki	36
3.1.3	Wybór backendu kompilatora	37
3.2	Procedura i narzędzia pomiarowe	37
3.2.1	Procedura pomiarowa	37
3.2.2	Narzędzia pomiarowe	38
3.2.3	Przykład wyznaczenia metryk	38
3.2.4	Kontrola warunków i zagrożenia dla trafności	39
4	Wyniki i analiza	40
4.1	Poprawność i wydajność	40
4.1.1	Weryfikacja poprawności numerycznej	40
4.1.2	Skalowanie wątkowe na pojedynczym węźle	41
4.2	Skalowanie i ograniczenia komunikacji	43
4.2.1	Skalowanie liczbą węzłów	43
4.2.2	Skalowanie wątkowe przy dziewięciu węzłach	45
4.2.3	Skalowanie rozmiaru zadania	45
4.2.4	Synteza: system ograniczony komunikacją	47
4.2.5	Zestawienie zbiorcze i powtarzalność	48
5	Podsumowanie i wnioski	50
5.1	Wnioski	50
5.2	Ograniczenia	51
5.2.1	Kierunki dalszych prac	51
	Bibliografia	53

Wstęp

Symulacje zjawisk opisanych równaniami różniczkowymi cząstkowymi należą do najczęściej spotykanych zadań obliczeniowych w nauce i inżynierii. Wśród nich szczególne miejsce zajmują obliczenia *stencilowe*, czyli takie, w których nowa wartość w każdym punkcie siatki jest funkcją wartości z punktów sąsiadujących.

Trójwymiarowe równanie przewodnictwa ciepła jest tradycyjnym przykładem takiego problemu. Jego jawna dyskretyzacja różnicowa ma dokładnie te cechy, które sprawiają, że obliczenia *stencilowe* są trudne do zrównoleglenia, czyli niską intensywność arytmetyczną oraz konieczność wymiany danych brzegowych między częściami dziedziny w każdym kroku czasowym [1]. Program symulujący równanie ciepła nadaje się dobrze do badania granic skalowalności. Choć wzorzec komunikacji jest tu prosty i ograniczony do wymiany z najbliższymi sąsiadami, to właśnie przy licznych węzłach każda taka wymiana obciąża to samo, współdzielone łącze sieciowe, co czyni problem dobrym obiektem do ujawniania ograniczeń komunikacyjnych klastra.

Równoległe programowanie systemów rozproszonych tradycyjnie realizuje się za pomocą interfejsu przekazywania komunikatów MPI (ang. *Message Passing Interface*), co daje wysoką wydajność kosztem znacznej złożoności kodu, gdyż programista jawnie zarządza podziałem danych, buforami wymiany oraz synchronizacją [2]. Alternatywę oferują języki i biblioteki oparte na modelu *PGAS* (ang. *Partitioned Global Address Space*) [3]. Język Chapel, rozwijany pierwotnie przez firmę Cray, jest jednym z bardziej rozwiniętych języków w tej grupie [4]. Umożliwia zapisywanie obliczeń rozproszonych w notacji zbliżonej do sekwencyjnej, z globalnym widokiem tablic, wysokopoziomowymi iteratorami równoległymi i gotowymi rozkładami dziedzin. Szczegóły komunikacji pozostawia warstwie wykonawczej, opartej na bibliotece GASNet.

Zbliżoną tematykę podjęto w pracy współautorskiej poświęconej równoległej symulacji dwuwymiarowego przewodnictwa ciepła na klastrze Raspberry Pi oraz porównaniu implementacji w językach Chapel, C# i Julia [5], podczas gdy niniejsza praca obejmuje przypadek trójwymiarowy na klastrze Pionier.

Celem niniejszej pracy było zaprojektowanie, zaimplementowanie i wdrożenie na klastrze programu symulującego równanie przewodnictwa ciepła w Chapel oraz

analiza jego poprawności i skalowalności.

Rozdział 1

Problem i wykorzystane technologie

1.1 Równanie ciepła i metoda różnic skończonych

1.1.1 Model fizyczny

Rozważanym zjawiskiem fizycznym jest dyfuzja ciepła w jednorodnym, izotropowym ośrodku wypełniającym trójwymiarowy sześcian jednostkowy $\Omega = [0, 1]^3$. Pole skalarne $u(\mathbf{x}, t)$ opisuje temperaturę w punkcie $\mathbf{x} = (x, y, z) \in \Omega$ w chwili $t \geq 0$. Ewolucję tego pola opisuje równanie różniczkowe cząstkowe znane jako równanie przewodnictwa ciepła, zwane też równaniem dyfuzji:

$$\frac{\partial u}{\partial t} = \alpha \nabla^2 u = \alpha \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2} \right), \quad (1.1)$$

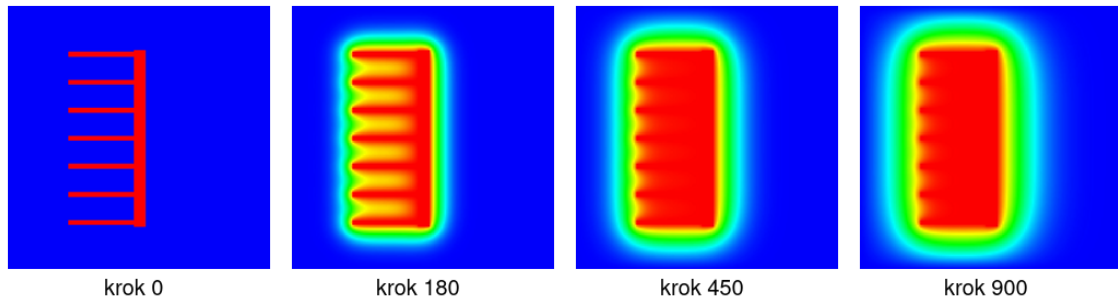
gdzie ∇^2 oznacza operator Laplace’a, natomiast $\alpha > 0$ to współczynnik dyfuzyjności cieplnej (ang. *thermal diffusivity*), wyrażony w jednostkach m^2/s . Wielkość α łączy przewodność cieplną ośrodka z jego pojemnością cieplną i gęstością. Im większa jej wartość, tym szybciej lokalne różnice temperatury ulegają wygładzeniu. W symulacji przyjęto znormalizowaną, bezwymiarową wartość α , nieodpowiadającą konkretnej wartości fizycznej. Wpływa ona jedynie na tempo wygładzania pola i nie ma wpływu na wyniki pomiarów wydajności.

1.1.2 Warunki początkowe i brzegowe

Rozwiązanie równania (1.1) jest jednoznaczne dopiero po zadaniu warunku początkowego oraz warunków brzegowych. Warunek początkowy przyjmuje jednorodne pole $u(\mathbf{x}, 0) = 1$ w całej dziedzinie, w którym umieszczono nagrzaną warstwę przy-

legającą do ściany $y = 0$, o podwyższonej wartości $u = 2$. Warstwa ta modeluje gorącą ścianę, od której ciepło rozchodzi się w głąb sześcianu.

Na brzegu dziedziny przyjęto warunki Dirichleta o ustalonych wartościach. Cała zewnętrzna warstwa komórek siatki nigdy nie jest aktualizowana i zachowuje swoje wartości początkowe przez cały czas symulacji. W praktyce oznacza to, że aktualizowane jest wyłącznie wnętrze dziedziny, a warstwa gorąca przy $y = 0$ pozostaje utrzymywana na poziomie $u = 2$, pełniąc rolę stałego źródła ciepła. Pozostałe ściany, utrzymywane na poziomie $u = 1$, działają jak zbiorniki ciepła o ustalonej temperaturze. Taki dobór warunków daje pole, które monotonnie ewoluuje i z czasem dąży do stanu ustalonego, z gradientem między gorącą a zimnymi ścianami. Rysunek 1.1 ilustruje jakościowo charakter tej ewolucji na dwuwymiarowym przykładzie. Podtrzymywane źródło ciepła stopniowo traci ostre krawędzie, a ciepło rozchodzi się w otoczenie. Dla czytelności wizualnej ilustracja ta posługuje się źródłem ciepła w kształcie grzebienia w przestrzeni dwuwymiarowej, które ukazuje ten sam mechanizm rozchodzenia się ciepła, lecz w układzie innym niż trójwymiarowa gorąca ściana rozważana w niniejszej pracy.



Rysunek 1.1: Ewolucja pola temperatury w dwuwymiarowej symulacji dyfuzji, wykonana dwuwymiarowym programem Chapel (dziedzina 160×160). Od lewej: początkowe, podtrzymywane źródło ciepła w kształcie grzebienia, a następnie stopniowe wygładzanie ostrych szczegółów i propagacja ciepła w otoczenie, aż do ukształtowania się gładkiego gradientu. Barwa odwzorowuje temperaturę. Błękit oznacza zimno, czerwień gorąco. Ta sama zasada obowiązuje w przypadku trójwymiarowym rozważanym w pracy.

1.1.3 Dyskretyzacja: jawna metoda różnic skończonych

Do rozwiązania równania (1.1) zastosowano jawną metodę różnic skończonych (ang. *Finite Difference Method*, FDM) z jawnym schematem Eulera w czasie i centralnymi różnicami drugiego rzędu w przestrzeni [6, 7]. Dziedzinę ciągłą zastępuje się regularną siatką kartezjańską o $n_x \times n_y \times n_z$ węzłach. Krok siatki w każdym z kierunków dobrany jest tak, aby węzły skrajne pokrywały się z brzegiem sześcianu

jednostkowego:

$$\Delta x = \frac{1}{n_x - 1}, \quad \Delta y = \frac{1}{n_y - 1}, \quad \Delta z = \frac{1}{n_z - 1}. \quad (1.2)$$

Wartość pola w węźle (i, j, k) w kroku czasowym n oznacza się przez $u_{i,j,k}^n$. Do przybliżenia pierwszej pochodnej czasowej użyto ilorazu różnicowego przedniego, a drugich pochodnych przestrzennych standardowych trzypunktowych ilorazów centralnych, na przykład dla kierunku x :

$$\frac{\partial^2 u}{\partial x^2}(x_i, y_j, z_k) \approx \frac{u_{i-1,j,k}^n - 2u_{i,j,k}^n + u_{i+1,j,k}^n}{(\Delta x)^2}, \quad (1.3)$$

i analogicznie dla kierunków y oraz z . Podstawienie tych przybliżeń do (1.1) i rozwiązanie względem wartości w następnym kroku prowadzi do jawnej formuły aktualizacji.

Dla zwięzłości zapisu wprowadza się współczynniki wagowe zależne od osi, łączące dyfuzyjność, krok czasowy oraz krok siatki:

$$a_x = \frac{\alpha \Delta t}{(\Delta x)^2}, \quad a_y = \frac{\alpha \Delta t}{(\Delta y)^2}, \quad a_z = \frac{\alpha \Delta t}{(\Delta z)^2}. \quad (1.4)$$

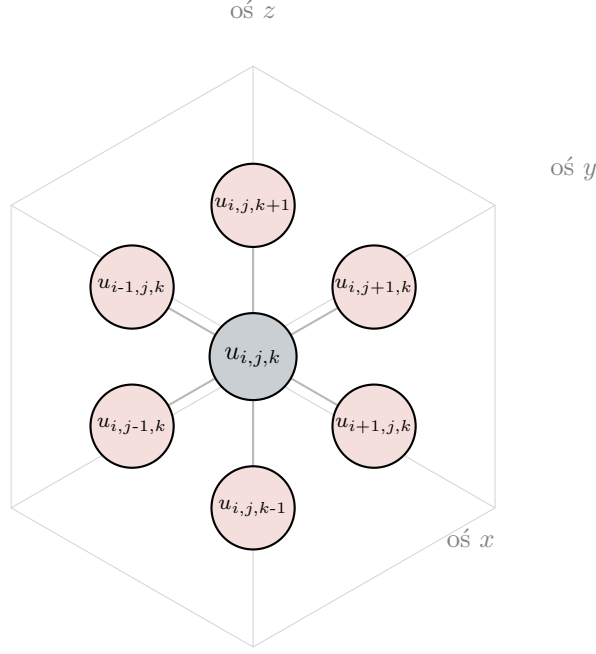
Przy tych oznaczeniach regułę aktualizacji można zapisać jako siedmiopunktowy stencil (ang. *7-point stencil*):

$$\begin{aligned} u_{i,j,k}^{n+1} = & u_{i,j,k}^n + a_x (u_{i-1,j,k}^n - 2u_{i,j,k}^n + u_{i+1,j,k}^n) \\ & + a_y (u_{i,j-1,k}^n - 2u_{i,j,k}^n + u_{i,j+1,k}^n) \\ & + a_z (u_{i,j,k-1}^n - 2u_{i,j,k}^n + u_{i,j,k+1}^n). \end{aligned} \quad (1.5)$$

Nowa wartość w każdym węźle zależy wyłącznie od wartości w tym węźle oraz w jego sześciu bezpośrednich sąsiadach z poprzedniego kroku czasowego, po dwóch wzdłuż każdej z trzech osi. Ten wzorec dostępu do pamięci, komórka centralna plus sześć sąsiadów, przedstawiono na rysunku 1.2. Zależność od stanu wyłącznie z poprzedniego kroku jest cechą schematu jawnego i czyni każdy krok czasowy w pełni równoległym względem węzłów siatki.

1.1.4 Stabilność numeryczna i warunek CFL

Zaletą schematów jawnych jest prostota i niski koszt pojedynczego kroku. Nie wymagają one rozwiązywania układów równań, w przeciwieństwie do schematów niejawnych. Ceną za tę prostotę jest jednak warunkowa stabilność. Krok czasowy Δt nie może być dowolnie duży [6, 8]. Zbyt duży krok powoduje, że błędy zaokrągleń i skła-



Rysunek 1.2: Siedmiopunktowy stencil trójwymiarowy. Nowa wartość w komórce centralnej $u_{i,j,k}$, zaznaczonej na niebiesko, jest ważoną kombinacją jej wartości bieżącej oraz sześciu bezpośrednich sąsiadów zaznaczonych na czerwono, wzdłuż osi x , y i z , zgodnie z równaniem (1.5).

dowe wysokoczęstotliwościowe rozwiązania numerycznego są wzmacniane w każdej iteracji zamiast tłumione, co prowadzi do gwałtownej, нефизycznej rozbieżności rozwiązania.

Analiza stabilności metodą von Neumanna [7, 6], polegająca na rozłożeniu błędu na składowe falowe i śledzeniu ich ewolucji krok po kroku, zastosowana dla schematu jawnego równania dyfuzji w trzech wymiarach prowadzi do warunku Couranta–Friedrichsa–Lewy’ego (CFL). Dla liniowego schematu o stałych współczynnikach współczynnik wzmocnienia pojedynczego modu Fouriera, określający, ile razy amplituda tego modu rośnie lub maleje w jednym kroku, wynosi

$$g = 1 - 4a_x \sin^2(\theta_x/2) - 4a_y \sin^2(\theta_y/2) - 4a_z \sin^2(\theta_z/2), \quad (1.6)$$

a żądanie $|g| \leq 1$ dla najbardziej niekorzystnego modu o naprzemiennym znaku między sąsiednimi węzłami redukuje się do nierówności $a_x + a_y + a_z \leq \frac{1}{2}$. Sumaryczna waga dyfuzyjna nie może przekraczać jednej drugiej, co wyznacza maksymalny dopuszczalny krok czasowy:

$$\Delta t_{\max} = \frac{1}{2\alpha \left(\frac{1}{(\Delta x)^2} + \frac{1}{(\Delta y)^2} + \frac{1}{(\Delta z)^2} \right)}. \quad (1.7)$$

Zależność (1.7) ma praktyczne konsekwencje, ponieważ dwukrotne zagęszczenie siatki wymusza czterokrotne zmniejszenie kroku czasowego, a więc czterokrotny wzrost liczby kroków potrzebnych do przebycia tego samego przedziału czasu fizycznego. Wraz z ośmiokrotnym wzrostem liczby węzłów siatki daje to duży wzrost całkowitego nakładu obliczeniowego, co jest jedną z przyczyn, dla których symulacje dużych siatek wymagają obliczeń rozproszonych.

1.2 Zrównoleglenie obliczeń stencilowych

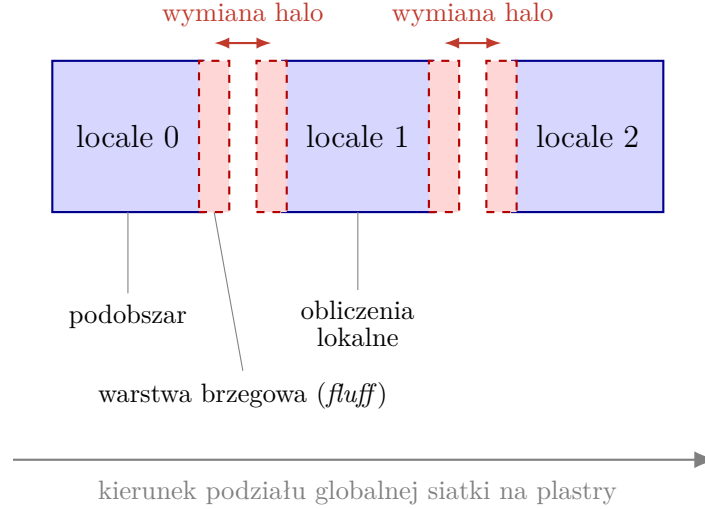
1.2.1 Dekompozycja dziedziny i komórki brzegowe

Obliczenia stencilowe należą do klasy problemów zrównoleglanych przez podział danych (ang. *data parallelism*). Ponieważ aktualizacja każdego węzła według (1.5) jest niezależna w obrębie jednego kroku czasowego, naturalną strategią jest dekompozycja dziedziny (ang. *domain decomposition*). Globalna siatka zostaje podzielona na rozłączne podobszary, z których każdy przypisany jest do osobnej jednostki obliczeniowej, w rozważanym przypadku do osobnego węzła klastra. Każdy węzeł oblicza aktualizację wyłącznie dla przydzielonych mu komórek.

Problem pojawia się na granicach podobszarów. Aby zaktualizować komórki leżące tuż przy krawędzi swojego podobszaru, węzeł potrzebuje wartości sąsiadów znajdujących się już w podobszarze sąsiedniego węzła. Rozwiązaniem jest otoczenie każdego lokalnego podobszaru dodatkową warstwą komórek brzegowych (ang. *ghost cells* lub *halo*), które przechowują kopie wartości z krawędzi sąsiednich podobszarów. W terminologii Chapel warstwa ta nosi nazwę *fluff*. Przyjmuje się ją o grubości jednej komórki w każdym kierunku, co odpowiada zasięgowi siedmiopunktowego stencilu. Rysunek 1.3 przedstawia schematycznie podział dziedziny na podobszary wraz z warstwami brzegowymi.

1.2.2 Wymiana brzegów jako jedyna komunikacja

W każdym kroku czasowym warstwy brzegowe muszą zostać odświeżone, zanim rozpocznie się faza obliczeń, gdyż przechowywane w nich wartości pochodzą z poprzedniego kroku sąsiedniego węzła. Ta wymiana danych z najbliższymi sąsiadami (ang. *nearest-neighbour* lub *halo exchange*) jest jedynym rodzajem komunikacji między węzłami wymaganym przez schemat jawny. W każdym kroku odbywa się ona na początku kroku, przed pętlą aktualizującą wnętrze dziedziny. Na więcej niż jednym locale wymiana ta jest obowiązkowa dla poprawności. Jej pominięcie sprawia, że komórki otoczki zachowują wartości z poprzedniego kroku, przez co obliczenie przebiega szybciej, lecz daje wynik cicho błędny.



Rysunek 1.3: Dekompozycja dziedziny w postaci plastrów (ang. *slab decomposition*) na trzy jednostki (*locale*). Globalna siatka jest cięta wzdłuż jednej osi, którą wskazuje strzałka na dole. Każdy podobszar, zaznaczony na niebiesko, otoczony jest warstwą komórek brzegowych *fluff*, zaznaczoną na czerwono linią przerywaną, przechowującą kopie krawędzi sąsiadów. Przed każdym krokiem obliczeń warstwy brzegowe są odświeżane w ramach wymiany z najbliższymi sąsiadami.

Prowadzi to do wyraźnego rozdzielenia każdego kroku czasowego na dwie fazy, fazę komunikacji, czyli odświeżenie warstw brzegowych, oraz fazę obliczeń, czyli wykonanie formuły (1.5) dla wszystkich komórek wnętrza.

1.2.3 Intensywność arytmetyczna i ograniczenia wydajności

Obliczenia stencilowe mają bardzo niską *intensywność arytmetyczną* (ang. *arithmetic intensity*) [1], czyli iloraz liczby wykonanych operacji zmiennoprzecinkowych do liczby bajtów przeniesionych z pamięci. Aktualizacja jednego węzła według (1.5) wymaga zaledwie kilkunastu operacji zmiennoprzecinkowych, sześciu odejmowań, trzech mnożeń przez wagi i kilku dodawań, a jednocześnie odczytu siedmiu wartości i zapisu jednej. Ponieważ liczba zmiennoprzecinkowa podwójnej precyzji zajmuje 8 bajtów, na każdy przeniesiony bajt przypada zaledwie ułamek operacji. Sąsiednie węzły współdzielą większość odczytów, lecz nawet przy idealnym wykorzystaniu pamięci podręcznej stosunek ten pozostaje niski.

W efekcie wydajność obliczeń stencilowych ogranicza nie moc procesora, lecz przepustowość transferu danych. W ujęciu modelu *roofline* [9], który wyznacza górną granicę wydajności na podstawie intensywności arytmetycznej zadania oraz przepustowości pamięci, punkt pracy obliczeń stencilowych leży w obszarze ograniczonym przepustowością pamięci, a nie szczytową wydajnością obliczeniową. Na pojedynczym węźle oznacza to ograniczenie przepustowością pamięci (ang. *memory-bandwidth bound*). Dodawanie kolejnych rdzeni obliczeniowych przynosi coraz mniej-

sze korzyści, gdy magistrala pamięci ulega nasyceniu. W obliczeniach rozproszonych, gdzie warstwy brzegowe muszą przejść przez sieć, zadanie staje się ograniczone komunikacją (ang. *communication bound*). O całkowitym czasie decyduje przepustowość i opóźnienie połączeń sieciowych między węzłami, a nie szybkość samych obliczeń.

Zależność ta ma jednak korzystną stronę. Objętość obliczeń w podobszarze rośnie proporcjonalnie do jego objętości, czyli jak N^3 dla sześcianu o boku N , podczas gdy objętość danych brzegowych podlegających wymianie rośnie proporcjonalnie do pola powierzchni granic, czyli jak N^2 . Stosunek nakładu obliczeń do nakładu komunikacji jest zatem rzędu $N^3/N^2 = N$ i rośnie liniowo z rozmiarem krawędzi siatki. Większe sześciany mają zatem korzystniejszy stosunek obliczeń do komunikacji, bo koszt komunikacji zostaje zamortyzowany większą liczbą obliczeń przypadających na każdy przesłany bajt.

1.2.4 Miary skalowalności

Do ilościowej oceny efektywności zrównoleglenia stosuje się standardowy aparat pojęciowy. *Przyspieszenie względne* (ang. *relative speedup*) przy użyciu p jednostek obliczeniowych definiuje się jako stosunek czasu wykonania programu na jednej jednostce do czasu wykonania tego samego programu na p jednostkach:

$$S(p) = \frac{T(1)}{T(p)}, \quad (1.8)$$

gdzie $T(1)$ jest czasem wykonania tego samego programu na jednej jednostce i stanowi punkt odniesienia. *Efektywność równoległą* (ang. *parallel efficiency*) definiuje się jako przyspieszenie znormalizowane liczbą jednostek:

$$E(p) = \frac{S(p)}{p} = \frac{T(1)}{p T(p)}. \quad (1.9)$$

Idealne, liniowe przyspieszenie odpowiada $S(p) = p$ oraz $E(p) = 1$, a w praktyce narzuty komunikacji, synchronizacji i nierównomiernego obciążenia sprawiają, że $E(p) < 1$. Efektywność zwykle maleje ze wzrostem p .

Wyróżnia się dwa rodzaje skalowania. *Skalowanie silne* (ang. *strong scaling*) polega na ustaleniu całkowitego rozmiaru problemu i zwiększaniu liczby jednostek obliczeniowych. Celem jest skrócenie czasu rozwiązania. Przy tym skalowaniu udział pracy przypadającej na jednostkę maleje, przez co narzuty komunikacyjne stają się względnie coraz istotniejsze. *Skalowanie słabe* (ang. *weak scaling*) polega na ustaleniu rozmiaru problemu przypadającego na jedną jednostkę i proporcjonalnym zwiększaniu zarówno liczby jednostek, jak i całkowitego rozmiaru problemu. Celem jest rozwiązywanie coraz większych zadań w stałym czasie.

Fundamentalne ograniczenie skalowania silnego formułuje *prawo Amdahla* [10]. Jeśli f oznacza sekwencyjną część obliczeń, której z natury nie da się zrównoleglić, to maksymalne osiągalne przyspieszenie przy p jednostkach wynosi

$$S(p) \leq \frac{1}{f + \frac{1-f}{p}}, \quad \lim_{p \rightarrow \infty} S(p) = \frac{1}{f}. \quad (1.10)$$

Nawet niewielki udział części sekwencyjnej narzuca twardą górną granicę przyspieszenia, niezależnie od liczby dostępnych jednostek. W obliczeniach stencilowych rolę nieusuwalnego narzutu pełni głównie komunikacja, wymiana warstw brzegowych oraz synchronizacja między krokami, co, w połączeniu z ograniczeniem przepustowości pamięci i sieci omówionym wcześniej, wyznacza granice skalowalności.

1.3 Chapel, PGAS i GASNet

1.3.1 Język Chapel i model PGAS

Implementację programu oparto na języku Chapel, języku programowania równoległego rozwijanym z naciskiem na wygodę programisty przy zachowaniu wysokiej wydajności na systemach wieloprocesorowych i rozproszonych [4, 11]. Chapel realizuje model *PGAS* [3, 12]. Idea PGAS polega na tym, że pamięć wszystkich węzłów tworzy jedną logicznie wspólną przestrzeń adresową, która jest jednak podzielona na partycje o jawnie określonym powinowactwie do węzłów. Programista operuje na danych za pomocą jednolitej, globalnej perspektywy (ang. *global-view*), tak jakby były one lokalne, natomiast środowisko uruchomieniowe automatycznie tłumaczy dostępy do danych zdalnych na operacje komunikacji sieciowej. W ten sposób PGAS łączy wygodę programowania w pamięci wspólnej, typową dla modelu pamięci współdzielonej, z kontrolą rozmieszczenia danych, którą daje model przekazywania komunikatów.

Chapel nie jest jedynym językiem tej grupy. Poza nim model PGAS realizują między innymi UPC (ang. *Unified Parallel C*), rozszerzenie języka C o globalne wskaźniki, Co-Array Fortran z notacją współmacierzy (ang. *co-arrays*), który wszedł do standardu Fortran 2008, a także języki projektowane od podstaw dla PGAS: X10 (IBM) i Fortress (Sun Microsystems). Rozwiązania te różnią się głównie składnią, modelem zadań i sposobem wyrażania lokalności, natomiast wspólne jest im oddanie programiście globalnego widoku danych przy jawnym oznaczaniu powinowactwa do węzłów [3, 12].

Centralnym pojęciem w Chapel jest *locale*, abstrakcja jednostki systemu dysponującej własną pamięcią oraz zdolnością do wykonywania obliczeń. W konfiguracji

wielowęzłowej stosowanej w niniejszej pracy każdemu locale odpowiada dokładnie jeden węzeł fizyczny klastra, w odwzorowaniu *jeden-do-jednego*. Wbudowana tablica `Locales` udostępnia wszystkie jednostki, a ich liczbę podaje stała `numLocales`. Dostęp do danych i przenoszenie obliczeń pomiędzy jednostkami wyraża się wprost, za pomocą konstrukcji `on`.

1.3.2 Iteratory równoległe i tablice globalne

Chapel oferuje dwa uzupełniające się rodzaje równoległości. Równoległość danych wyraża pętla `forall`, która zleca wykonanie ciała iteracji dla wszystkich elementów dziedziny współbieżnie, pozostawiając środowisku uruchomieniowemu podział pracy na zadania i przypisanie ich do wątków. W programie to właśnie pętla `forall` nad dziedziną wnętrza (listing 2.9) realizuje fazę obliczeń stencila. Gdy pętla iteruje po dziedzinie rozproszonej, jej iteracje wykonują się na tych węzłach, do których przypisane są odpowiednie elementy, co zapewnia lokalność obliczeń bez jawnego zarządzania nią przez programistę.

Równoległość zadaniową między węzłami wyraża konstrukcja `coforall ...on loc`, która tworzy osobne zadanie dla każdego locale i umieszcza jego wykonanie na odpowiednim węźle. W programie wzorzec ten wykorzystywany jest między innymi do równoległego zapisu klatek wynikowych, gdzie każdy węzeł niezależnie zapisuje swój lokalny podobszar. Listing 1.1 ilustruje ten idiom.

```
1 coforall loc in Locales do on loc {  
2   // lokalny fragment tablicy globalnej  
3   const ld = field.localSubdomain();  
4   // kopia danych bez komunikacji zdalnej  
5   var block: [ld] real = field[ld];  
6   // ... zapis / obliczenia lokalne na fragmencie ...  
7 }
```

Listing 1.1: Równoległość zadaniowa między węzłami: każdy locale wykonuje operację na swoim lokalnym podobszarze danych.

Tablice w Chapel definiowane są nad *dziedzinami* (ang. *domains*), obiektami opisującymi zbiór indeksów. Aby tablica została rozproszona pomiędzy węzły, jej dziedzinę wyposaża się w *odwzorowanie dziedziny* (ang. *domain map*) lub inaczej *dystrybucję*. Najprostszą dystrybucją jest `Block`, dzieląca zakres indeksów na w przybliżeniu równe, przylegające bloki i przypisująca je kolejnym węzłom. Na potrzeby obliczeń stencilowych Chapel udostępnia wyspecjalizowaną dystrybucję `Stencil` z modułu `StencilDist`, która poza podziałem danych utrzymuje opisaną wcześniej warstwę *fluff* i odświeża ją jawnym wywołaniem `updateFluff()` [13]. W programie dziedzinę tworzy się wywołaniem `stencilDist.createDomain(..., fluff=(1,1,1))`,

ustalając grubość otoczki na jedną komórkę w każdym kierunku zgodnie z zasięgiem stencila (1.5).

1.3.3 Konstrukcje równoległości i synchronizacja

Oprócz pętli `forall` i konstrukcji `coforall...on` Chapel udostępnia pomocnicze mechanizmy równoległości i synchronizacji, które pozwalają wyrażać wzorce nie dające się zapisać samym podziałem danych.

Równoległość na poziomie instrukcji realizują `begin` i `cobegin`. Instrukcja `begin` uruchamia pojedyncze zadanie w tle i nie blokuje dalszego wykonywania bieżącego wątku, a `cobegin` wykonuje każde z podanych zdań równoległe i czeka na zakończenie ich wszystkich. Pozwala to na zagnieżdżanie zadań i tworzenie struktur, w których kilka operacji przeplata się w czasie. Listing 1.2 ilustruje ten idiom.

```
1 cobegin {  
2     writeTask();    // zadanie A  
3     readTask();     // zadanie B, wykonuje się równoległe z A  
4 }  
5 // oba zadania kończą się zanim sterowanie przejdzie dalej
```

Listing 1.2: Równoległe uruchamianie zadań za pomocą `cobegin`.

Konstrukcja `on` przenosi wykonanie instrukcji na wskazany locale. Wewnątrz bloku słowo kluczowe `here` oznacza locale, na którym blok jest aktualnie wykonywany, czyli ten wskazany przez `on`, a nie locale, który to wykonanie zlecił. Pozwala to odczytać właściwości locale wykonującego bez jawnego odwoływania się do elementów tablicy `Locales`. Listing 1.3 pokazuje ten idiom.

```
1 on Locales[1] do  
2     writeln("działam na: ", here.id, " z ", here.maxTaskPar, "  
    watkami");
```

Listing 1.3: Przeniesienie wykonania na konkretny locale z użyciem `here`.

Do synchronizacji producenta z konsumentem służą zmienne `single` (tak zwane zmienne synchronizujące). Zapis do takiej zmiennej wstrzymuje wykonanie, dopóki nie zostanie odczytana, a odczyt wstrzymuje je, dopóki nie zostanie zapisana. Dają one semantykę jednorazowego przekazania wartości między zadaniami bez jawnych zamków (ang. *locks*). Listing 1.4 przedstawia ten mechanizm.

```

1 var s: single int;           // zmienna synchronizujaca
2 begin { s = computeResult(); } // producent
3 const v = s;                 // konsument czeka na wartosc

```

Listing 1.4: Zmienna synchronizująca jako kanał jednorazowy między zadaniami.

Dla bezpiecznej manipulacji stanem współdzielonym Chapel udostępnia typy `atomic`, które gwarantują niepodzielność operacji odczytu, zapisu i dodawania. Typowe zastosowanie to liczniki globalne, sumy częściowe i sekwencjonowanie dostępu do zasobów, bez ryzyka wyścigu. Listing 1.5 pokazuje typowe użycie.

```

1 var counter: atomic int;
2 counter.add(1);           // atomowe dodawanie
3 const n = counter.read(); // atomowy odczyt

```

Listing 1.5: Operacje atomowe na liczniku współdzielonym.

Wreszcie, dla typowych operacji redukcyjnych Chapel oferuje wyrażenia redukcji, które pozwalają zredukować tablicę lub pętlę do pojedynczej wartości przy użyciu operatorów takich jak `+`, `min` czy `max`, przy czym samo dzielenie pracy i akumulacja wyniku pozostają w rękach środowiska uruchomieniowego. Listing 1.6 ilustruje redukcję sumy.

```

1 var s = + reduce A;

```

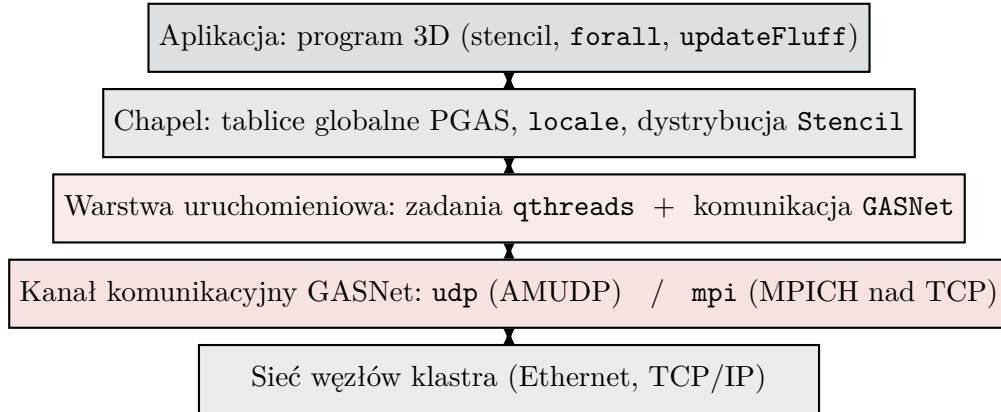
Listing 1.6: Suma wszystkich elementów tablicy A.

1.3.4 Warstwa zadań `qthreads` oraz warstwa komunikacji GASNet

Wykonywanie zadań w obrębie pojedynczego locale obsługuje scheduler zadań. W zastosowanej konfiguracji jest to biblioteka `qthreads` (`CHPL_TASKS=qthreads`), lekki system wątków użytkownika efektywnie obsługujący dużą liczbę drobnoziarnistych (ang. *fine-grained*) zadań generowanych przez pętlę `forall` [14]. Maksymalną liczbę wątków, jakie warstwa zadań może uruchomić równolegle na danym węźle, udostępnia własność `here.maxTaskPar`. Wyznacza ona stopień równoległości fazy obliczeń w obrębie węzła. Jest to liczba wątków obliczeniowych na locale, a nie liczba locale, i jest raportowana przez program przy starcie.

Za komunikację między węzłami odpowiada z kolei warstwa *GASNet*, przenośna biblioteka uruchomieniowa dostarczająca podstawowych operacji komunikacji jednostronnej (ang. *one-sided*) oraz komunikatów aktywnych, na których Chapel opiera realizację dostępu do pamięci zdalnej i operacji `on` [15, 16]. Uaktywnia się ją ustawieniem `CHPL_COMM=gasnet`. W zastosowanej konfiguracji przyjęto tryb segmentu

CHPL_GASNET_SEGMENT=everything, w którym cała pamięć procesu jest udostępniana dla komunikacji zdalnej, co upraszcza rozmieszczanie tablic rozproszonych. Warstwową strukturę całego stosu, od kodu aplikacji, przez model globalnej tablicy PGAS, aż po sprzęt sieciowy, przedstawia rysunek 1.4.



Rysunek 1.4: Warstwowa architektura wykonania. Kod aplikacji operuje na globalnych tablicach PGAS języka Chapel. Ich dostępy zdalne i wymiana warstw brzegowych są realizowane przez warstwę uruchomieniową, w której zadania `qthreads` wykonują się lokalnie, a komunikacja `GASNet` zachodzi między węzłami. Warstwa ta korzysta z kolei z wybranego kanału komunikacyjnego odwzorowującego operacje na fizyczną sieć klastra.

1.3.5 Kanały komunikacyjne GASNet

GASNet abstrahuje sprzęt sieciowy poprzez wymienne *kanały komunikacyjne* (ang. *conduits*), moduły implementujące operacje komunikacji na konkretnej technologii transportu. W ramach niniejszej pracy porównywano dwa kanały komunikacyjne dostępne na klastrze zbudowanym z węzłów połączonych zwykłą siecią Ethernet. Kanał `udp`, oparty na warstwie AMUDP (ang. *Active Messages over UDP*), realizuje komunikację po protokole UDP i jest uruchamiany za pomocą programu rozruchowego `amudprun`. Jego zaletą jest prosta konfiguracja, a wadą podatność na przeciążenie przy intensywnej, drobnoziarnistej wymianie danych. Kanał `mpi` przenosi komunikację GASNet na istniejącą implementację MPI, w zastosowanej konfiguracji opartej na MPICH nad TCP, i jest uruchamiany programem `gasnetrun_mpi`. Zapewnia on stabilniejszy i wydajniejszy transport dla podtrzymywanych, regularnych wzorców komunikacji, jakie generuje cykliczna wymiana warstw brzegowych w opisywanym programie.

Wybór kanału komunikacyjnego ma istotne znaczenie dla rozproszonej symulacji. Jak wykazano w podrozdziale 1.2, przy skalowaniu na wiele węzłów zadanie jest ograniczone komunikacją, a jedyną komunikacją w kroku jest wymiana *halo* realizowana przez `updateFluff()`. O osiąganym skalowalności decydują w praktyce właściwości warstwy komunikacyjnej, czyli przepustowość, opóźnienie i odporność

na przeciążenie. Dlatego dobór kanału komunikacyjnego jest jednym z głównych pytań badawczych w tej pracy.

Rozdział 2

Klaster i implementacja

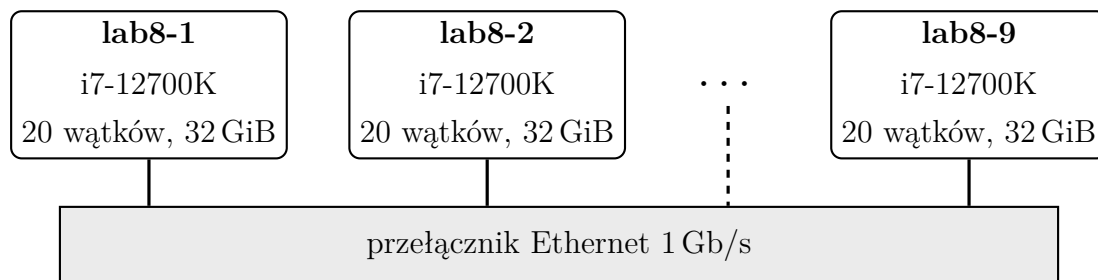
2.1 Architektura klastra i środowisko

2.1.1 Sprzęt i topologia

Docelowym środowiskiem uruchomieniowym jest rzeczywisty klaster *Pionier*¹ w pracowni komputerowej Wydziału Matematyki i Informatyki UMK, złożony z dziewięciu jednorodnych węzłów fizycznych (*Pionier-lab8-1* ... *Pionier-lab8-9*) połączonych przełącznikiem Ethernet o przepustowości 1 Gb/s. Wszystkie węzły są identyczne, a każdy wyposażony jest w procesor Intel Core i7-12700K (12 rdzeni, 20 wątków sprzętowych), 32 GiB pamięci RAM DDR4-3200 oraz kartę sieciową Intel I219-V (sterownik `e1000e`, 1 Gb/s) i pracuje pod systemem Ubuntu (jądro 6.8.0-111-generic). Węzły mają adresy z podsieci 192.168.128.0/24, a dostęp do nich odbywa się bezpośrednim poleceniem `ssh <ip>`. Jednorodność środowiska, czyli ta sama architektura procesora, dystrybucja i wersja bibliotek na każdym węźle, upraszcza dystrybucję plików wykonywalnych, gdyż obraz zbudowany na jednym węźle działa bez zmian na pozostałych. W eksperymentach skalowania każdemu locale odpowiadał jeden węzeł fizyczny, co pozwoliło zbadać skalowanie w funkcji liczby węzłów aż do dziewięciu.

Topologię środowiska przedstawiono na rysunku 2.1. Wszystkie węzły dzielą jedno łącze o przepustowości 1 Gb/s, które jest wąskim gardłem komunikacji rozproszonej, bo każda wymiana komórek brzegowych między sąsiednimi fragmentami dziedziny przechodzi przez ten sam przełącznik.

¹Klaster został zakupiony w ramach projektu PIONIER-LAB (nr POIR.04.02.00-30-A005/16) sfinansowanego przez Unię Europejską w ramach Funduszy Europejskich Programu Operacyjnego Inteligentny Rozwój.



Rysunek 2.1: Topologia jednorodnego, dziewięciowęzłowego klastra Pioneer. Wszystkie identyczne węzły (*Pionier-lab8-1* ... *Pionier-lab8-9*) połączone są jednym przełącznikiem 1 Gb/s, który stanowi współdzielone i zarazem najwęższe łącze komunikacji rozproszonej. Środowisko budowane jest raz na dowolnym węźle, a pliki wykonywalne rozprowadzane na pozostałe.

2.1.2 Konfiguracja kompilatora Chapel

Kompilator Chapel w wersji 2.9.0 zbudowano ze źródeł na jednym węźle [17]. Kluczowe parametry środowiska są zapisane w pliku `chplconfig`, który skrypt `distribute-chapel.sh` aktualizuje jeszcze przed wywołaniem `make`. Dzięki temu każdy węzeł otrzymuje pliki wykonywalne zbudowane z identyczną konfiguracją. Zawartość istotnej części pliku konfiguracyjnego przedstawia listing 2.1.

```

1 CHPL_COMM=gasnet
2 CHPL_COMM_SUBSTRATE=$CONDUIT          # udp (domyslnie) albo mpi
3 CHPL_LLVM=none                        # backend C, nie LLVM
4 CHPL_TARGET_CPU=$TARGET_CPU          # native

```

Listing 2.1: Fragment pliku `chplconfig` generowanego przed budową kompilatora.

Warstwą komunikacyjną jest GASNet (`CHPL_COMM=gasnet`), co umożliwia pracę wielolokalową, a kompilacja odbywa się backendem C (`CHPL_LLVM=none`) zamiast LLVM. Wybór backendu dotyczy wyłącznie środowiska, w którym kompilowany jest program Chapel, ponieważ gotowy plik wykonywalny nie ma dynamicznych powiązań do bibliotek LLVM. Jako że pomiary nie wykazały różnicy w wydajności między backendem C i LLVM (tabela 3.2), wybrano backend C, prostszy i szybszy w budowie. Ustawienie `CHPL_TARGET_CPU=native` pozwala generować kod korzystający z pełnego zestawu instrukcji procesora, co jest bezpieczne, gdyż wszystkie węzły mają ten sam procesor. Z tej konfiguracji wynikają też domyślne wartości `CHPL_TASKS=qthreads`, `CHPL_GASNET_SEGMENT=everything`, `CHPL_LOCALE_MODEL=flat` oraz `CHPL_TARGET_MEM=jemalloc`, a program startowy przyjmuje `amudprun` dla kanału `udp` albo `gasnetrun_mpi` dla kanału `mpi`.

2.1.3 Dwa kanały komunikacyjne GASNet

Warstwę GASNet można zbudować na dwóch różnych kanałach transportowych (*conduit*), wybieranych parametrem `--conduit` [16]. Kanał **udp** realizuje protokół AMUDP nad UDP. Nie ma dodatkowych zależności i jest szybki w czystej sieci lokalnej. Kanał **mpi** realizuje GASNet mpi-conduit za pośrednictwem biblioteki MPICH w trybie `ch3:sock` (zwykły TCP), którego mechanizm kontroli przepływu zapewnia odporność na utratę pakietów i zjawisko incast [18, 2]. Ze względu na to, że wybór kanału zapisany jest w pliku `chplconfig`, dla każdego kanału utrzymywany jest osobny, w pełni skompilowany katalog Chapel (ścieżki `.../chapel` oraz `.../chapel-mpi`).

Biblioteka MPICH w wersji 4.2.3 została zbudowana ze źródeł skryptem `build-mpi.sh` do prefiksu niewymagającego uprawnień administratora, z linią konfiguracji przedstawioną w listingu 2.2. Budowa ze źródeł zapewnia na każdym węźle *identyczną* wersję i konfigurację MPI, w szczególności urządzenie `ch3:sock` (zwykły TCP), którego kontroli przepływu nie gwarantują gotowe pakiety systemowe dostarczające zwykle OpenMPI w domyślnej konfiguracji.

```
1 ./configure --prefix=$PREFIX --disable-fortran \  
2     --with-device=ch3:sock --disable-static
```

Listing 2.2: Linia konfiguracji budowy MPICH ze źródeł.

2.1.4 Awaria kanału udp pod obciążeniem incast

Kanał `udp` ma istotną wadę przy obciążeniu wielowęzłowym. AMUDP przenosi komunikację bezpośrednio po UDP, protokole bez kontroli przepływu, w którym nadawca nie otrzymuje sygnału o przeciążeniu odbiorcy. Wymiana halo jest wzorcem wielu-do-jednego (*incast*). W jednym kroku wiele locale wysyła jednocześnie fragmenty brzegowe do tego samego odbiorcy, nasycając jego bufor odbiorczy UDP i powodując gubienie pakietów. Mechanizm kontroli kongestii AMUDP wykrywa to i przerywa działanie błędem `ECONGESTION` [19]. Manifestuje się to nagłym przerwaniem przebiegu, niezależnym od poprawności obliczeń, już na wczesnym etapie przy większych kosztach rozproszonych na wiele węzłów.

Kanał `mpi` jest na to odporny: MPICH w trybie `ch3:sock` korzysta ze zwykłego TCP, którego kontrola przepływu w razie przeciążenia odbiorcy cofa nadawcę, a straty pakietów obsługuje retransmisją. Wspólny przełącznik Ethernet, który dla kanału `udp` jest punktem awarii przy incast, dla kanału `mpi` działa jako zwykły most z kontrolą przepływu. Kosztem jest niewielki narzut protokołu TCP, nieistotny wobec zyskanej stabilności, dlatego kanał `mpi` przyjęto jako produkcyjny.

2.1.5 Uruchamianie i samoopisujące się logi

Dla każdego pliku wykonywalnego skrypt `compile-and-distribute.sh` (funkcja `gen_run_script`) generuje osobny skrypt uruchomieniowy. Ustawia on zmienne `CHPL_HOME` oraz `PATH`, domyślnie przyjmuje liczbę wątków na locale równą `$(nproc)`, a przebieg rejestruje do logu o nazwie zawierającej znacznik czasu (`logs/<bin>-YYYYmmdd-HHMMSS.log`). Zasadniczy fragment logiki uruchomieniowej pokazano w listingu 2.3.

```
1 set -o pipefail
2 {
3     echo "[cfg] threadsPerLocale(requested)=
4         $CHPL_RT_NUM_THREADS_PER_LOCALE numLocales=${NUM_LOCALES}"
5     ./${bin} -nl ${NUM_LOCALES} "$@" 2>&1
6 } | tee "$LOG"
```

Listing 2.3: Blok uruchomieniowo-rejestrujący ze skryptu wrappera.

Log jest samoopisujący dzięki dwóm liniom `[cfg]`. Pierwsza, pochodząca ze skryptu, ma postać `[cfg] threadsPerLocale(requested)=N numLocales=M` i podaje wartości żądane. Druga, pochodząca z programu, ma postać `[cfg] locale <id> maxTaskPar=<n>` i podaje efektywną liczbę wątków na locale po ograniczeniu jej przez `qthreads` do możliwości sprzętu [14]. Rozróżnienie to jest istotne przy analizie wydajności, ponieważ w przypadku nadsubskrypcji (*oversubscription*) liczba żądana może przewyższać liczbę faktycznie dostępnych rdzeni. Konfiguracja programu startowego pochodzi ze wspólnego pliku `run-env.sh`, który definiuje funkcję `set_run_env` ustawiającą to środowisko niezależnie od kanału. Plik ten jest jedynym źródłem tej konfiguracji, wspólnym dla wygenerowanych skryptów uruchomieniowych oraz dla skryptu pomiarowego `bench.sh`. Dla kanału `mpi` funkcja ustawia `MPIRUN_CMD="mpirun -n %N -hosts <hosts> <install>/mpi-iface-wrap.sh %C"`, a dla kanału `udp` zmienne `GASNET_SSH_SERVERS` oraz `GASNET_MASTERIP`.

Ostatnim elementem środowiska jest skrypt `mpi-iface-wrap.sh` odpowiedzialny za przypięcie interfejsu sieciowego per ranga na węzłach wielosieciowych. Wybiera on lokalny adres IPv4 z właściwej podsieci klastra (`192.168.128.0/24`), pomijając interfejsy wirtualne, czyli mostki `docker0` i `virbr0`, podsieci `172.16/12` oraz adresy `libvirt 192.168.122.*`, eksportuje `MPICH_INTERFACE_HOSTNAME`, po czym uruchamia program. Bez tego kroku transport `ch3:sock` może rozgłosić nieroutowalny adres interfejsu, a wywołanie `MPI_Init` kończy się błędem „No route to host”.

2.1.6 Procedura wdrożenia krok po kroku

Elementy opisane w poprzednich podrozdziałach tworzą procedurę wdrożenia, od zbudowania środowiska aż po uruchomienie przebiegu. Mimo jednorodności węzłów proces ten jest bardziej złożony niż lokalna kompilacja, gdyż znaczenie mają kolejność kroków i miejsce ich wykonania, a na wszystkich węzłach potrzebne są spójne ścieżki oraz identyczne katalogi Chapel i MPICH. Procedurę podzielono na cztery etapy.

W pierwszym etapie na dowolnie wybranym węźle, co umożliwia jednorodność klastra, budowane jest środowisko wykonawcze, czyli kompilator Chapel w wersji 2.9.0 (skryptem `distribute-chapel.sh`) oraz biblioteka MPICH w wersji 4.2.3 w trybie `ch3:sock` (skryptem `build-mpi.sh`).

W drugim etapie katalog ze skompilowanymi programami i zależnościami Chapel jest pakowany do archiwum i rozsyłany pod identyczną ścieżkę bezwzględną na każdym węźle (listing 2.4). Warstwa uruchomieniowa Chapel kompilowana jest ze ścieżką `rpath` wskazującą wprost katalog `CHPL_HOME`, toteż każda rozbieżność lokalizacji uniemożliwia dynamiczne odnalezienie bibliotek współdzielonych. Tak samo jeden katalog MPICH, zbudowany raz, trafia pod tę samą ścieżkę na wszystkich węzłach, co zapewnia zgodność ABI wymaganą przez `mpi-conduit`.

```
1 tar czf chapel-2.9.0-built.tar.gz -C "$CHPL_HOME/.." chapel-2.9.0
2 for h in $HOSTS; do
3     scp chapel-2.9.0-built.tar.gz "$h:$DEST/"
4     ssh "$h" "tar xzf $DEST/chapel-2.9.0-built.tar.gz -C $DEST"
5 done
```

Listing 2.4: Spakowanie i rozesłanie zbudowanego katalogu pod identyczną ścieżkę.

W trzecim etapie kompilowany jest sam program i rozprowadzany na węzły skryptem `compile-and-distribute.sh`. Dla każdego programu kopiowane są oba pliki, startowy `<bin>` oraz właściwy obraz wykonywalny `<bin>_real`, niezbędne na każdym węźle do uruchomienia wielolokalowego. Skrypt generuje przy tym dedykowany skrypt uruchomieniowy, który ustawia `CHPL_HOME` i `PATH` zgodnie z rozesłanym katalogiem.

W czwartym etapie przebieg uruchamiany jest przez wygenerowany skrypt, który wywołuje właściwy program startowy GASNet (`gasnetrun_mpi` dla kanału `mpi` albo `amudprun` dla kanału `udp`) i rozdziela locale na węzły. Powodzenie tego etapu zależy od obu niezmienników z poprzednich kroków, czyli identycznej ścieżki `CHPL_HOME` w `rpath` oraz identycznych katalogów Chapel i MPICH pod tą samą ścieżką na wszystkich węzłach. Naruszenie któregośkolwiek z nich ujawnia się dopiero przy ładowaniu bibliotek lub starcie procesów zdalnych, a więc już po udanej kompilacji.

2.1.7 Uruchamianie eksperymentu

Pojedynczy przebieg pomiarowy obsługują dwa skrypty generowane na węźle głównym. Oba tworzy skrypt `compile-and-distribute.sh`, który sam nie jest generowany. Kompiluje on program z wybranym kanałem komunikacyjnym, rozprowadza pliki wykonywalne na węzły z pliku `hostfile` i zapisuje na miejscu skrypt uruchomieniowy `run-heat3d.sh` oraz agregujący `aggregate-heat3d.sh`. Nie buduje przy tym kompilatora Chapel ani biblioteki MPICH, a jedynie kompiluje program i rozsyła go na już przygotowane węzły. Skrypt uruchomieniowy wywołuje właściwy program z ustawionym środowiskiem Chapel i GASNet, przekazując mu swoje argumenty jako parametry `config` programu. Liczbę locale zapisuje w sobie na etapie generowania, zaś liczbę wątków na locale ustala zmienna `CHPL_RT_NUM_THREADS_PER_LOCALE`. Skrypt agregujący zbiera rozproszone zrzuty klatek na węzeł główny i uruchamia post-procesor `aggregate3d.chpl`, opcjonalnie renderując film.

Kluczowe flagi skryptu `compile-and-distribute.sh` zestawiono w tabeli 2.1. Pełną listę podaje opcja `--help`.

Tabela 2.1: Wybrane flagi skryptu `compile-and-distribute.sh`.

Flaga	Wartość domyślna	Znaczenie
<code>-f, --hostfile</code>	brak (wymagane)	plik z adresami węzłów, master jako pierwszy
<code>-c, --conduit</code>	<code>udp</code>	kanał transportu: <code>udp</code> lub <code>mpi</code>
<code>-d, --dir</code>	<code>/home/<user></code>	zdalny katalog instalacji
<code>-u, --user</code>	<code>chapel</code>	użytkownik SSH
<code>-p, --port</code>	<code>22</code>	port SSH
<code>-V, --chapel-version</code>	<code>2.9.0</code>	wersja Chapel (nazwa katalogu)
<code>-m, --mpi-dir</code>	<code><dir>/mpi</code>	prefiks MPICH (kanał <code>mpi</code>)
<code>-o, --output</code>	<code>heat3d</code>	nazwa pliku wykonywalnego programu
<code>-s, --skip-compile</code>	wyłączony	pomiń kompilację, użyj istniejących plików wykonywalnych

Typowy przebieg pomiarowy składa się z trzech poleceń wykonywanych na węźle głównym, pokazanych kolejno w listingach 2.5, 2.6 i 2.7. Najpierw budowa i dystrybucja, z kanałem `mpi` i wskazanym katalogiem Chapel:

```
1 ./compile-and-distribute.sh -f hosts.txt -c mpi -d /home/chapel/
  workspace
```

Listing 2.5: Budowa i rozesłanie plików wykonywalnych na klaster.

Następnie uruchomienie programu na zadanej liczbie węzłów, z parametrami dziedziny i liczby kroków przekazanymi jako flagi `config`, oraz z żadaną liczbą wątków na locale:

```
1 CHPL_RT_NUM_THREADS_PER_LOCALE=4 ./run-heat3d.sh --nx=1000 --
  numSteps=100
```

Listing 2.6: Uruchomienie przebiegu pomiarowego.

Na koniec, po serii przebiegów, scalenie rozproszonych zrzutów i opcjonalny render filmu:

```
1 ./aggregate-heat3d.sh --render=true
```

Listing 2.7: Agregacja rozproszonych fragmentów i wizualizacja.

Pojedyncze polecenia z listingów 2.5, 2.6 i 2.7 odpowiadają jednej konfiguracji. Systematyczne serie pomiarowe realizuje skrypt `bench.sh`, który automatyzuje ten przebieg. Powtarza on każdą konfigurację domyślnie dziesięciokrotnie, wykonuje przebiegi sekwencyjnie i przemiana badany parametr w ramach serii `threads`, `cube` oraz `nodes`, a osobna seria `llvm` porównuje backendy kompilatora. Skrypt korzysta z tego samego pliku `run-env.sh` co skrypty wygenerowane oraz zapisuje logi z liniami `[cfg]` dla każdego przebiegu wraz z zestawieniem zbiorczym serii. Wyniki porównania backendów z serii `llvm` przedstawiono w podrozdziale 3.1.3 oraz tabeli 3.2.

2.2 Struktura programu i krok obliczeniowy

2.2.1 Dziedzina rozproszona z otoczką

Podstawą implementacji jest trójwymiarowa dziedzina rozproszona między locale za pomocą rozkładu `StencilDist` [13], utrzymującego otoczkę (*fluff*) o grubości jednej komórki. Definicję dziedziny oraz jej wnętrza przedstawia listing 2.8.

```
1 const domain3D = stencilDist.createDomain({0..<nx,0..<ny,0..<nz},
2                                           fluff=(1,1,1));
3 const interior = domain3D.expand(-1);
```

Listing 2.8: Definicja dziedziny rozproszonej z jednokomórkową otoczką oraz jej wnętrza.

Wnętrze `interior` powstaje przez zmniejszenie dziedziny o jedną komórkę w każdym kierunku (`expand(-1)`). Obejmuje ono wyłącznie te komórki, dla których siedmiopunktowy stencil ma komplet sąsiadów wewnątrz dziedziny globalnej. Komórki brzegowe, wyłączone z wnętrza, pełnią rolę stałego warunku brzegowego typu Dirichleta.

2.2.2 Pojedynczy krok obliczeniowy

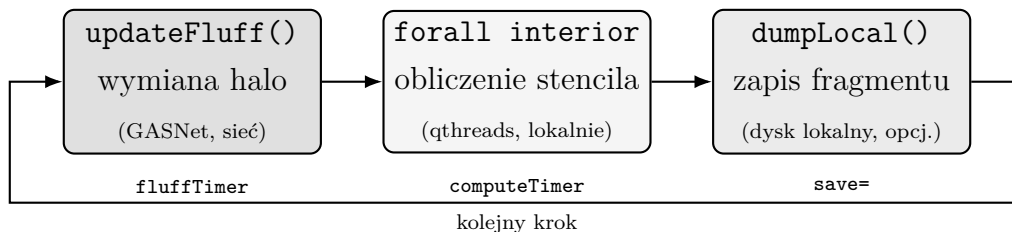
Pojedynczy krok czasowy realizuje procedura `stepOnce`, przedstawiona w listingu 2.9. Dzieli ona krok na dwie fazy, wymiany halo i obliczenia stencila, mierzone osobnymi stoperami.

```

1 proc stepOnce(ref dst, ref src, ref fluffTimer, ref computeTimer) {
2   fluffTimer.start();
3   src.updateFluff(); // halo
4   fluffTimer.stop();
5   computeTimer.start();
6   forall (i,j,k) in interior do
7     dst[i,j,k] = src[i,j,k]
8                 + ax*(src[i-1,j,k]-2*src[i,j,k]+src[i+1,j,k])
9                 + ay*(src[i,j-1,k]-2*src[i,j,k]+src[i,j+1,k])
10                + az*(src[i,j,k-1]-2*src[i,j,k]+src[i,j,k+1]);
11   computeTimer.stop();
12 }
```

Listing 2.9: Procedura pojedynczego kroku: wymiana halo, a następnie obliczenie stencila.

Metoda `updateFluff()` kopiuje komórki brzegowe każdego locale do otoczek sąsiadów przez GASNet. Jest to jedyna komunikacja kroku, obowiązkowa dla poprawności w konfiguracji wielolokalowej. Stanowi ona jednocześnie główne wąskie gardło obliczeń rozproszonych. Przebieg pojedynczego kroku wraz z fazą zapisu ilustruje rysunek 2.2.



Rysunek 2.2: Potok pojedynczego kroku czasowego programu. Faza komunikacyjna (`updateFluff`) poprzedza lokalne obliczenie stencila, po którym następuje opcjonalny zapis fragmentu na dysk lokalny. Każda faza mierzona jest osobnym stoperem.

2.2.3 Ewolucja implementacji i operator zamiany buforów

Pętla czasowa używa dwóch pól, u oraz un , czyli bieżącego i poprzedniego. Po każdym kroku trzeba je zamienić, aby następny krok czytał najnowszy stan. Dalej opisano, jak ten mechanizm zmieniał się w kolejnych wersjach programu i dlaczego w wariancie głównym użyto wbudowanego operatora zamiany Chapel. Pozwala to też sprawdzić, czy sposób zamiany buforów wpływa na wydajność.

Najpierw zbudowano wersję jednowymiarową (`1d.chpl`) jako weryfikację pomysłu, a następnie dwuwymiarową (`2d.chpl`). Obie stanowiły kolejne kroki rozwojowe prowadzące do przypadku trójwymiarowego. W obu tych wariantach po każdym kroku obliczeniowym wykonywano jawną zamianę buforów zapisem przedstawionym w listingu 2.10:

```
1  un <=> u;
```

Listing 2.10: Zamiana buforów operatorem `<=>` stosowana w wariantach 1D i 2D oraz w wcześniejszej wersji 3D.

Po każdej aktualizacji pole bieżące zamieniało się zatem z polem poprzednim, tak by kolejny krok czytał z bufora przechowującego najnowszy stan.

Wczesna wersja trójwymiarowa, zanim wprowadzono opisany w podrozdziale 2.3 rozproszony zapis fragmentów, renderowała klatki bezpośrednio w trakcie symulacji za pomocą modułu graficznego `Render3D`, wywoływanego w każdym kroku. Takie rozwiązanie umieszczało kosztowne operacje wejścia-wyjścia i graficzne na ścieżce krytycznej obliczeń, co utrudniało czysty pomiar czasu pojedynczego kroku. W ramach optymalizacji usunięto moduł renderujący ze ścieżki obliczeniowej. Stan pola zapisywany jest odtąd w postaci binarnych fragmentów przez procedurę `dumpLocal` opisaną w podrozdziale 2.3, a renderowanie przeniesiono do jednolokalowego post-procesora `aggregate3d.chpl` uruchamianego poza pętlą czasową.

Wariant `3d.chpl`, stosowany we wszystkich pomiarach, zachowuje stałe role argumentów, zawsze wywołując `stepOnce(u, un, ...)`, a po każdym kroku wykonuje wspomnianą wyżej zamianę operatorem `<=>`. Jego pętla główna przedstawiona jest w listingu 2.11. Dla porównania zaimplementowano również wariant ping-pong (`3d_pingpong.chpl`), w którym role bufora docelowego oraz źródłowego są naprzemiennie, co wyraża warunek `writeToU = (step % 2 == 1)`, gdzie w krokach nieparzystych wynik pisany jest do u , a w parzystych do un , dzięki czemu po parzystej liczbie kroków wynik znajduje się ponownie w pierwotnym buforze.

```

1 for step in 1..numSteps {
2   var fluffT, computeT, saveT: stopwatch;
3   stepOnce(u, un, fluffT, computeT); // stale: dst=u, src=un
4   un <=> u;                          // zamiana buforow
5   if step % dumpEvery == 0 then
6     dumpLocal(un, frame);           // un zawsze trzyma
        najnowszy stan
7 }

```

Listing 2.11: Pętla główna wariantu 3d.chpl: stałe role buforów i jawna zamiana operatorem `<=>` po każdym kroku.

Wariant wykorzystany w dalszej części pracy opiera się na operatorze `<=>`, wbudowanym operatorze zamiany tablic w Chapel, ponieważ dla rozkładu `StencilDist` nie powoduje on kopiowania elementów. Implementacja operatora próbuje najpierw ścieżkę `doiOptimizedSwap`, która zamienia wewnętrzne wskaźniki danych obu tablic w czasie stałym $O(1)$, bez ruchu danych i bez komunikacji sieciowej. Jako że `StencilDist`, podobnie jak `BlockDist`, implementuje tę ścieżkę, zamiana sprowadza się do przepięcia wskaźników lokalnych buforów na każdym locale, a jedynym dodatkowym kosztem jest to, że `doiOptimizedSwap` wykonuje się jako `coforall loc in Locales do on loc { ... }`, co wprowadza jedną barierę synchronizacyjną między locale po każdym kroku.

Mechanizm ten ilustruje listing 2.12, gdzie operator `<=>` rozprasza zamianę do metody `doiOptimizedSwap` rozkładu, a dla `StencilDist` metoda ta zamienia lokalne wskaźniki danych na każdym locale w czasie stałym.

Ponieważ zamiana `<=>` nie przenosi danych, jej koszt nie przekracza kosztu wspomnianej bariery, czyli ułamka czasu pojedynczego kroku, a sam krok pozostaje zdominowany przez wymianę halo oraz zapis fragmentu na dysk. Sposób zarządzania buforami nie ma więc znaczącego wpływu na wydajność, a wariant ping-pong pozostawiono jako punkt odniesienia pomijający barierę zamiany.

2.2.4 Pomiar czasu i raportowanie

Pomiar czasu rozdzielono na osobne stopery mierzące poszczególne fazy programu. Stoper `initTimer` obejmuje fazę przygotowania, czyli alokację tablic, inicjalizację gorącej warstwy oraz obliczenie kroku czasowego `dt` i współczynników, a jego wynik raportowany jest jako *Initialization time*. Stoper `computeTimer` obejmuje całą pętlę czasową i raportowany jest jako *Execution time*, a wewnątrz niej każdy krok mierzą trzy dalsze stopery, obejmujące wymianę halo w metodzie `updateFluff`, właściwe obliczenia oraz zapis fragmentu na dysk. Ich odczyty wypisują po każdym kroku linię `step <n> updateFluff=<s> s compute=<s> s save=<s> s`. Ponadto

```

1 // operator <=> (modules/internal/ChapelArray.chpl)
2 inline operator <=>(ref x: [?xD], ref y: [?yD]) {
3     var hasSwapped = false;
4     if x.eltype == y.eltype then
5         if Reflection.canResolveMethod(x._value, "doiOptimizedSwap", y.
6             _value) then
7             hasSwapped = x._value.doiOptimizedSwap(y._value);
8         if !hasSwapped then // fallback: kopiowanie
9             element po elemencie
10             forall (a,b) in zip(x, y) do a <=> b;
11 }
12
13 // doiOptimizedSwap dla StencilDist (modules/dists/StencilDist.
14 // chpl)
15 proc StencilArr.doiOptimizedSwap(other: this.type) {
16     if canDoOptimizedSwap(other) {
17         coforall (locarr1, locarr2) in zip(this.locArr, other.locArr)
18             do on locarr1 {
19                 locarr1.myElems <=> locarr2.myElems; // O(1): zamiana
20                 // lokalnych wskaźników
21                 locarr1.locRAD <=> locarr2.locRAD;
22             }
23         return true;
24     }
25     return false;
26 }

```

Listing 2.12: Fragment implementacji operatora `<=>` oraz metody `doiOptimizedSwap` rozkładu `StencilDist` w źródłach Chapelu (uproszczony).

logger `commLog` wypisuje jednorazowo linię `[comm] <bytes> B/step across <N> locale(s)` oraz przyrostowe liczby operacji `put/get/on/amo` na każdy krok, co pozwala powiązać obserwowany czas z faktyczną objętością komunikacji.

Efektywną liczbę wątków na locale wypisuje poniższa pętla, przez co linie są uporządkowane według identyfikatora locale.

```
1 for loc in Locales do on loc do
2   writeln("[cfg] locale ", here.id, " maxTaskPar=", here.maxTaskPar
   );
```

Listing 2.13: Wypisanie efektywnej liczby wątków dla każdego locale.

2.2.5 Parametry konfiguracyjne

Zachowanie programu sterowane jest zbiorem stałych konfiguracyjnych `config const`, które można nadpisać z linii poleceń bez rekompilacji. Domyślne wartości zestawiono w tabeli 2.2.

Tabela 2.2: Stałe konfiguracyjne programu i ich wartości domyślne (wariant 3d.chp1).

Parametr	Wartość domyślna	Znaczenie
<code>nx, ny, nz</code>	20	wymiary siatki w trzech osiach
<code>numSteps</code>	100	liczba kroków czasowych
<code>alpha</code>	0,25	współczynnik dyfuzji
<code>hotThickness</code>	2	liczba gorących warstw wzdłuż osi <i>y</i>
<code>debug</code>	false	tryb diagnostyczny
<code>dumpDir</code>	frames	katalog zrzutów
<code>dumpEvery</code>	100	co ile kroków zapisywać klatkę
<code>compress</code>	true	kompresja zrzutów <code>gzip</code>
<code>trackMem</code>	false	śledzenie zużycia pamięci
<code>commLog</code>	false	rejestrwanie statystyk komunikacji

2.2.6 Raportowanie przebiegów

Przebiegi raportowane są w celu, by jednocześnie śledzić postęp na bieżąco, zachować trwały zapis do dalszej analizy i jednoznacznie wiązać log z konfiguracją, w której powstał. W tym celu wyjście uruchamianego programu przechwytywane jest poleceniem `tee` działającym po ustawieniu `set -o pipefail`, dzięki czemu kod wyjścia potoku odzwierciedla wynik programu, a nie polecenia `tee`.

Nazwa pliku logu zawiera znacznik czasu w postaci `logs/<bin>-YYYYmmdd-HHMMSS.log`, przez co kolejne przebiegi tego samego pliku wykonywalnego nie nadpisują się nawzajem i pozostają rozróżnialne w czasie.

2.3 Buforowanie, zapis i agregacja danych

2.3.1 Zapis fragmentów na dysk lokalny

Zapis stanu pola do dalszej wizualizacji zaprojektowano tak, aby ciężkie operacje wejścia-wyjścia nie obciążały ścieżki krytycznej obliczeń. Procedura `dumpLocal`, pokazana w listingu 2.14, powoduje, że każde locale zapisuje wyłącznie własny fragment przestrzeni na lokalny system plików danego węzła, bez globalnego gromadzenia danych w jednym miejscu. Dzięki temu zapis jest w pełni rozproszony i skaluje się z liczbą węzłów.

```
1 proc dumpLocal(ref field, frameIdx: int) throws {
2   coforall loc in Locales do on loc {
3     const ld = field.localSubdomain();
4     const base = dumpDir + "/frame_" + frameIdx:string
5                   + "_loc_" + here.id:string + ".bin";
6     var block: [ld] real = field[ld];
7     // zapis nagłówka i surowego bloku (opcjonalnie przez gzip)
8   }
9 }
```

Listing 2.14: Rozproszony zapis: każdy locale zapisuje własny podblok lokalnie.

Powstałe w wyniku pliki noszą nazwę zgodnie ze schematem `frame_<idx>_loc_<localeId>.bin`, lub `.bin.gz` przy włączonej kompresji, w katalogu `dumpDir`. Nagłówek zawiera indeks klatki, identyfikator locale, sześć globalnych granic i sześć granic lokalnych, po których następuje surowy blok liczb typu `real`, a jego przykład dla klatki o indeksie 100 zapisanej przez locale 1 w dziedzinie 100^3 podzielonej między dwa locale wzdłuż osi x przedstawiono w tabeli 2.3.

Tabela 2.3: Przykładowa zawartość nagłówka pliku zrzutu.

Pole	Wartość
indeks klatki	100
locale	1
granice globalne	$[0, 99] \times [0, 99] \times [0, 99]$
granice lokalne	$[50, 99] \times [0, 99] \times [0, 99]$
blok danych	<code>real(64)</code> , $50 \cdot 100 \cdot 100 = 500000$ wartości

Kompresja realizowana jest przez przekierowanie strumienia do uruchomionego procesu `gzip -c > file.gz`. Kompresja `gzip` jest bezstratna niezależnie od gładkości symulowanego pola i zmniejsza rozmiar danych około 190-krotnie, co czyni częste zrzucanie klatek przystępnym pod względem zajętości dysku. Parametr `dumpEvery` pełni rolę bramki (`step % dumpEvery == 0`), a licznik klatek zwiększa się przy każdym faktycznym zrzucie. Przed rozpoczęciem pętli każdy `locale` wykonuje `rmTree` i `mkdir` na katalogu `dumpDir`, aby wyeliminować pozostałości po wcześniejszych przebiegach. W przebiegach z licznymi zrzutami klatek pojawiać się może błąd `ENOSPC` (numer 28), czyli problem dyskowy, inny niż sieciowy `ECONGESTION`. Rozróżnienie obu bywa istotne przy diagnostyce nieudanych przebiegów.

2.3.2 Agregacja rozproszonych fragmentów i wizualizacja

Skoro dane zapisywane są w postaci rozproszonych podbloków, konieczny jest osobny etap ich scalenia. Realizuje go post-procesor `aggregate3d.chpl`. Dla każdej klatki alokuje on pełne, gęste pole, odnajduje wszystkie pliki `frame_<f>_loc_*.bin*`, pliki skompresowane rozpakowuje poleceniem `gunzip -c`, a następnie odczytuje każdy podblok, którego nagłówek odzwierciedla format zapisany przez `dumpLocal`, i rozmieszcza go we właściwym miejscu pełnego pola. Kod rekonstrukcji pola pokazano w listingu 2.15.

```
1 const blockDom = {lo[0]..hi[0], lo[1]..hi[1], lo[2]..hi[2]};
2 var block: [blockDom] real;
3 r.readBinary(block);
4 full[blockDom] = block;
```

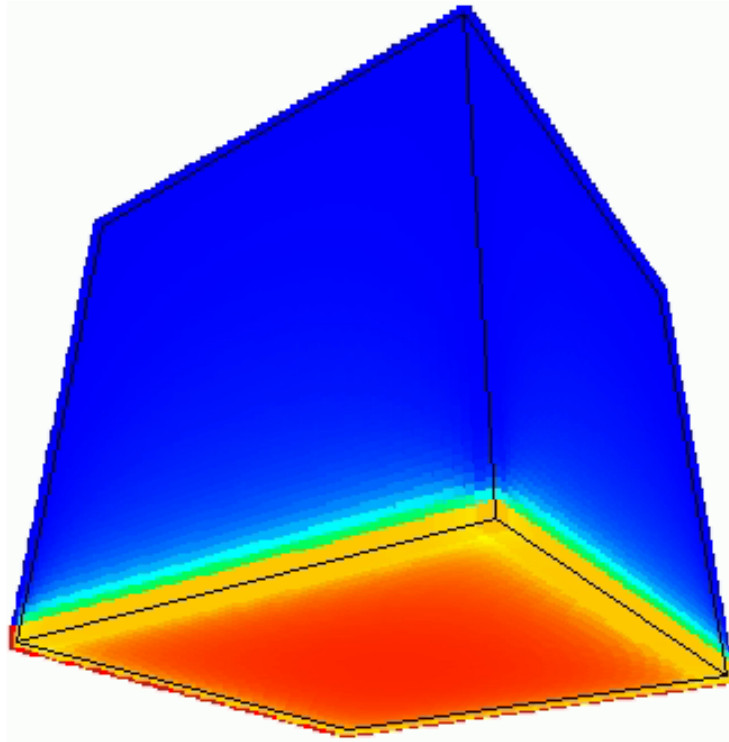
Listing 2.15: Rekonstrukcja pełnego pola: odczyt podbloku i rozmieszczenie go we właściwym miejscu.

Po zrekonstruowaniu pełnego pola dla danej klatki wywoływana jest procedura `renderFrame(full)`, przy czym podział na rozproszony zapis i scentralizowaną agregację jest zamierzony, aby komunikacja nie obciążała czasu obliczeń.

Ostatnim etapem, uruchamianym wyłącznie z parametrem `--render=true`, jest renderowanie zdefiniowane w module `ImageUtils.chpl`, w którym kolejne klatki kodowane są do pliku wideo o nazwie `movieName` z użyciem `mediaPipe`, poprzez klatki w formacie BMP. Zastosowano mapę barw HSV odwzorowującą ciepło od niebieskiego do czerwonego, gdzie odcień wyznaczany jest jako $240(1 - t)$ stopni, przy czym $t = (T - T_{\min}) / (T_{\max} - T_{\min})$ jest temperaturą znormalizowaną do przedziału $[0, 1]$. Wartość 240 odpowiada kolorowi niebieskiemu przy $t = 0$, a 0 kolorowi czerwonemu przy $t = 1$.

Trójwymiarowy renderer rysuje wyłącznie sześć zewnętrznych ścian sześcianu w rzucie perspektywicznym z obrotem kontrolowanym parametrami `rotX`, `rotY`, `camDist` oraz `cubeScale`. Zaimplementowano usuwanie ścian tylnych (*back-face culling*), Z-bufor, dylatację punktów (`pointSize`) oraz czarne krawędzie sześcianu kreślone algorytmem Bresenhama. Krawędzie podlegają temu samemu usuwaniu ścian tylnych, stąd spośród dwunastu rysowane są tylko te należące do widocznych ścian. Widoczne ściany próbkowane są o jedną komórkę do wnętrza, aby pominąć stałe krawędzie brzegowe.

Sześcian renderowany jest pod kątem, w widoku 3/4, od strony gorącej ściany $y = 0$. Ponieważ pięć ścian utrzymywanych jest w stałej temperaturze zimnej, a jedna w gorącej, render uwidacznia podtrzymywaną gorącą ścianę w kolorze czerwonym oraz gradient temperatury na widocznych ścianach bocznych, opadający ku zimnym krawędziom. Wynik dla dziedziny 64^3 przedstawiono na rysunku 2.3.



Rysunek 2.3: Render 3D pola temperatury dla dziedziny 64^3 , wykonany wbudowanym rendererem wokselowym programu z modułu `ImageUtils.chpl` i pokazany pod kątem od strony gorącej ściany. Widoczna jest podtrzymywana gorąca ściana $y = 0$ (czerwona, $T = 2$) oraz gradient na ścianach bocznych opadający ku zimnym ścianom ($T = 1$). Barwa odwzorowuje temperaturę w przedziale $[1, 2]$ zgodnie z mapą HSV od błękitu do czerwieni.

Opisana architektura oddziela operacje wejścia-wyjścia i komunikację zbiorową od właściwej pętli obliczeniowej poprzez rozproszony zapis fragmentów bez globalnego gromadzenia, ich późniejsza agregacja poza ścieżką krytyczną oraz opcjonalna wizualizacja. Jedynym elementem komunikacyjnym tej pętli pozostaje wymiana halo omówiona w podrozdziale 2.2. Pełny kod źródłowy programu, skrypty uruchomieniowe oraz narzędzia analizy stanowią załącznik elektroniczny do niniejszej pracy, co umożliwia odtworzenie opisanych eksperymentów.

Rozdział 3

Metodologia badań

3.1 Scenariusze i metryki

3.1.1 Scenariusze badawcze

Badania podzielono na dwie grupy, czyli weryfikację poprawności oraz pomiary wydajności i skalowania. Weryfikacja poprawności obejmowała sprawdzenie, że wynik obliczeń jest niezależny od liczby locale, na które rozdzielono dziedzinę, pod warunkiem prawidłowej wymiany warstw brzegowych. Jako niezmiennik kontrolny wykorzystano sumę wartości pola w stanie końcowym, raportowaną przez program w wierszu `final field: min=... max=... sum=....`. Ze względu na to, iż pole przyjmuje wartości w przedziale $[1, 2]$, a jego średnia jest bliska jedności, suma jest w przybliżeniu równa liczbie komórek siatki.

Pomiary wydajności obejmowały cztery grupy scenariuszy, zestawione w tabeli 3.1:

- **skalowanie wątkowe w obrębie węzła:** jeden locale (`-nl 1`), zmienna liczba wątków 1/2/4/8/16, przeprowadzone dla kostki 1000^3 na węźle Pionierlab8-1 o 20 wątkach.
- **skalowanie wątkowe przy 9 locale:** kostka 1000^3 , ustalona liczba locale, zmienna liczba wątków na locale.
- **skalowanie po liczbie węzłów:** kostka 1000^3 , 16 wątków na locale, liczba locale od 1 do 9.
- **skalowanie rozmiaru zadania:** 9 locale, 16 wątków na locale, krawędź kostki od 125 do 1000.

Skalowanie wątkowe w zakresie węzła i skalowanie po wielu węzłach rozdzielono świadomie, ponieważ to dwa różne mechanizmy równoległości, z których jeden

Tabela 3.1: Grupy scenariuszy pomiarowych. Liczbę wątków w pierwszej grupie oznaczono symbolem p , liczbę locale symbolem L , a krawędź kostki symbolem N .

Scenariusz	Zmienna	Ustalone	Środowisko
Wątki, 1000 ³	$p \in \{1,2,4,8,16\}$	$L=1$, $N=1000$, 10 kroków	węzeł Pionier-lab8-1 (20 wątków)
Wątki, 9 locale	$p \in \{1,2,4,8,16\}$	$L=9$, $N=1000$, 100 kroków	klaster (9 węzłów)
Węzły	$L \in \{1, \dots, 9\}$	$p=16$, $N=1000$, 100 kroków	klaster (9 węzłów)
Rozmiar kostki	$N \in \{125, \dots, 1000\}$	$p=16$, $L=9$, 100 kroków	klaster (9 węzłów)

współdzieli pamięć przez wątki jednego locale, a drugi rozprasza dane między wiele locale połączonych komunikacją sieciową. Zestawienie obu perspektyw pozwala zidentyfikować czynnik ograniczający wydajność.

Górna granica 16 wątków w powyższych scenariuszach wynika z architektury węzłów oraz z domyślnego zachowania warstwy wykonawczej Chapel. Procesor Intel Core i7-12700K jest układem hybrydowym, w którym występuje 8 rdzeni wydajnościowych (*P-core, performance core*) z wielowątkowością współbieżną, co daje 16 rdzeni logicznych, oraz 4 rdzeni energooszczędnych (*E-core, efficiency core*) bez wielowątkowości, łącznie 12 rdzeni fizycznych i 20 wątków logicznych. Skrypt uruchomieniowy żąda liczby wątków na locale równej wynikowi polecenia `nproc`, czyli w przypadku klastra Pionier jest to 20. Warstwa `qthreads`, korzystająca z biblioteki `hwloc`, przy domyślnym ustawieniu `CHPL_RT_USE_PU_KIND=performance` wykorzystuje jednak wyłącznie jednostki wykonawcze rdzeni wydajnościowych i pomija rdzenie energooszczędne, co na tych węzłach daje 16 dostępnych wątków logicznych. Żądanie 20 wątków przekracza tę liczbę, co sprawia, że warstwa wykonawcza ogranicza ją do 16 i zgłasza komunikat `Reduced numThreadsPerLocale=20 to 16 to prevent oversubscription` [17]. Każdy węzeł pracuje zatem efektywnie na 16 wątkach i tę wartość ustalono jako górną we wszystkich eksperymentach. Z tego samego powodu skalowanie wątkowe w obrębie węzła prowadzono w wartościach 1/2/4/8/16.

We wszystkich przebiegach za współczynnik dyfuzji przyjęto $\alpha = 0,25$, a gorącą warstwę startową o grubości `hotThickness=2` płaszczyzn inicjalizowano wzdłuż osi y przy ścianie $y = 0$. Krok czasowy dobierany jest automatycznie z warunku (1.7), z zastosowaniem współczynnika bezpieczeństwa obniżającego krok o 2 % poniżej granicy stabilności:

$$\Delta t = 0,98 \cdot \Delta t_{\max}, \quad (3.1)$$

co dla najbardziej niekorzystnego modu, o naprzemiennym znaku między sąsiednimi węzłami, daje współczynnik wzmocnienia (1.6) równy $g = -0,96$. Ujemny znak oznacza jedynie, że amplituda tego modu zmienia znak w każdym kroku, natomiast warunek $|g| < 1$ gwarantuje, że jej moduł maleje. Błędy nie narastają, lecz są tłumione, toteż schemat pozostaje stabilny mimo błędów zaokrągleń. Dla największej badanej kostki $N = 1000$ daje to $\Delta t \approx 6,55 \cdot 10^{-7}$.

3.1.2 Metryki

Podstawową mierzoną wielkością jest *czas wykonania* T , rozumiany jako czas trwania całej pętli czasowej, mierzony zegarem `computeTimer` w głównym locale (locale 0). Ponieważ krok czasowy zawiera barierę synchronizacyjną wymiany halo, wszystkie locale kończą krok w tym samym czasie, przez co pomiar z locale 0 jest reprezentatywny dla całego przebiegu. Czas ten nie obejmuje inicjalizacji, mierzonej osobno zegarem `initTimer`.

Na podstawie czasu wykonania zdefiniowano standardowe miary skalowania silnego, takie jak przyspieszenie względne $S(p) = T(1)/T(p)$ oraz wydajność równoległą $E(p) = S(p)/p$, zdefiniowane już wzorami (1.8) i (1.9), gdzie $T(1)$ jest czasem wykonania konfiguracji bazowej z jednym wątkiem albo jednym locale. W roli p dla skalowania wielowątkowego przyjęto *efektywną* liczbę wątków, czyli wartość `maxTaskPar` po ewentualnym ograniczeniu przez warstwę wątkową, a nie liczbę żadaną. Do oceny wydajności obliczeń niezależnie od rozmiaru zadania posłużono się *przepustowością obliczeniową* wyrażoną w miliardach aktualizacji komórek na sekundę,

$$\Pi = \frac{N^3 \cdot K}{T \cdot 10^9}, \quad (3.2)$$

gdzie $N^3 \cdot K$ jest łączną liczbą aktualizacji komórek, T jest czasem wykonania w sekundach, a czynnik 10^9 jest prefiksem giga zamieniającym aktualizacje na giga-aktualizacje. Π wyrażone jest w giga-aktualizacjach komórek na sekundę, co odpowiada konwencji metryki GUPS z systemu HPC Challenge [20]. Dla oszacowania roli komunikacji wprowadzono *udział komunikacji* w czasie pojedynczego kroku,

$$\text{comm_frac} = \frac{t_{\text{fluff}}}{t_{\text{fluff}} + t_{\text{compute}}}, \quad (3.3)$$

gdzie t_{fluff} i t_{compute} są średnimi czasami wymiany halo oraz obliczeń. Wielkość `comm_frac` przyjmuje wartości bliskie jedności, gdy w wykonaniu programu dominuje udział komunikacji, oraz bliskie zeru w przypadku, gdy obliczenia zajmują więcej czasu. Dodatkowo rejestrowano objętość wymiany halo w bajtach na krok, obliczaną analitycznie przez procedurę `haloBytesPerStep` jako łączną liczbę komórek

warstw brzegowych wszystkich locale pomnożoną przez rozmiar elementu.

3.1.3 Wybór backendu kompilatora

Chapel generuje kod wynikowy za pomocą backendu C (`CHPL_LLVM=none`) albo przez backend LLVM (`CHPL_LLVM=system`). Aby sprawdzić, czy wybór backendu wpływa na wydajność, ten sam program zbudowano w obu wariantach i porównano na dziewięciu węzłach klastra Pionier. Każdy wariant uruchomiono dziesięciokrotnie dla kostki 100^3 , przez 100 kroków, z warstwą komunikacyjną `mpi-conduit`. Backend LLVM korzystał z systemowej instalacji LLVM w wersji 19.1.7. Wyniki zestawiono w tabeli 3.2.

Tabela 3.2: Porównanie backendu kompilatora C (`CHPL_LLVM=none`) i LLVM (`CHPL_LLVM=system`, LLVM 19.1.7) dla kostki 100^3 na dziewięciu węzłach klastra Pionier, 100 kroków, 10 powtórzeń. Kolumny *mediana*, *średnia*, *min* i *max* dotyczą czasu wykonania, a kolumny *updateFluff* i *compute* podają medianę czasu danej fazy na krok.

Backend	mediana [s]	średnia [s]	min [s]	max [s]	updateFluff [s/krok]	compute [s/krok]
C (none)	20,015	19,995	19,883	20,096	0,1966	0,00185
LLVM 19.1.7 (system)	19,910	19,908	19,842	19,962	0,1955	0,00185

Mediany czasu wykonania różnią się o 0,105 s, czyli około 0,5 %, co mieści się w granicach rozrzutu między powtórzeniami. Wymiana buforów halo zajmuje około 99 % kroku, a same obliczenia około 1 %, więc udział komunikacji `comm_frac` według (3.3) jest bliski jedności. Backend kompilatora wpływa wyłącznie na lokalny kod obliczeniowy, którego czas jest w obu wariantach praktycznie taki sam i stanowi niewielką część kroku, dlatego nie przekłada się na całkowity czas wykonania. Z tego powodu we wszystkich dalszych pomiarach używano backendu C (`CHPL_LLVM=none`), gdyż skraca to czas budowy kompilatora i zmniejsza rozmiar instalacji.

3.2 Procedura i narzędzia pomiarowe

3.2.1 Procedura pomiarowa

Każdą konfigurację uruchamiano dziesięciokrotnie, a jako wartość reprezentatywną przyjmowano *medianę* czasu wykonania. Wybór ten wynika z faktu, iż pojedyncze przebiegi bywają zaburzone przez chwilowe zdarzenia systemowe, takie jak rywalizacja o rdzenie, przejściowe ograniczenie taktowania procesora czy aktywność innych procesów, które zawyżają czas, lecz nie odzwierciedlają właściwości badanej konfiguracji. Mediana jest odporna na takie wartości odstające, podczas gdy średnia

zostaje przez nie zawyżona. Przykładem jest jedna z serii jednowęzłowych, w której trzy z dziesięciu przebiegów były wolniejsze o około pół sekundy (około 5 % czasu pojedynczego przebiegu). Skutkowało to podwyższonym odchyleniem standardowym przy praktycznie niezmięnionej medianie.

Przebiegi danej serii wykonywano sekwencyjnie, przy użyciu skryptu `bench.sh`, opisanego w podrozdziale 2.1.7, po jednym naraz, aby uniknąć wzajemnej rywalizacji o pamięć i pasmo. Wszystkie pomiary wydajności prowadzono na środowisku Chapel 2.9 z warstwą komunikacyjną w wariantcie `mpi-conduit`. Liczbę wątków na locale ustawiano zmienną środowiskową `CHPL_RT_NUM_THREADS_PER_LOCALE`. Pomiary prowadzono bez zapisu klatek na dysk, ustawiając `dumpEvery` powyżej liczby kroków, dzięki czemu w mierzonej pętli nie zachodził żaden zapis, a operacje wejścia-wyjścia nie wpływały na czas kroku. Konfigurację każdego przebiegu zapisywano bezpośrednio w logu w postaci linii `[cfg]` opisanych w podrozdziale 2.1.7.

3.2.2 Narzędzia pomiarowe

Czasy mierzono wbudowanymi w język stoperami (`stopwatch`), osobno dla inicjalizacji, całej pętli czasowej oraz, w każdym kroku, dla wymiany halo, obliczeń i zapisu. Objętość i liczbę operacji komunikacyjnych GET, PUT, zdalnych i atomowych rejestrowano za pomocą modułu diagnostyki komunikacji GASNet, włączanego przełącznikiem `commLog`. Wypisywany raz wiersz `[comm]` podaje analityczną objętość wymiany halo na krok. Zebrane logi zostały przeanalizowane oraz zaprezentowane w postaci wykresów pudełkowych z użyciem skryptów napisanych w języku Python w kombinacji z biblioteką `matplotlib`.

3.2.3 Przykład wyznaczenia metryk

Dla uporządkowania powyższych definicji poniżej przedstawiono sposób wyznaczenia metryk pochodnych na jednym scenariuszu. Rozważono skalowanie wątkowe kostki 1000^3 na węźle Pionier. Jednowątkowa konfiguracja bazowa osiągnęła medianę czasu wykonania $T(1) = 9,812\text{ s}$, natomiast optymalna konfiguracja ośmiowątkowa $T(8) = 8,094\text{ s}$. Zgodnie z (1.8) przyspieszenie wynosi $S(8) = 9,812/8,094 \approx 1,212$, a wydajność równoległa według (1.9), $E(8) = 1,212/8 \approx 0,152$. Przepustowość obliczeniowa dla tej konfiguracji, przy $K = 10$ krokach, wynosi $1000^3 \cdot 10 = 10^{10}$ aktualizacji komórek w czasie $T(8) = 8,094\text{ s}$ co daje $10^{10}/8,094 \approx 1,24 \cdot 10^9$ aktualizacji na sekundę, a po podzieleniu przez 10^9 $\Pi \approx 1,24$ miliarda aktualizacji komórek na sekundę.

3.2.4 Kontrola warunków i zagrożenia dla trafności

Wiarygodność wniosków zależy od kontroli czynników zakłócających. W toku badań zidentyfikowano i ograniczono kilka takich czynników.

Różnorodność wyników czasowych pochodzi ze współdzielenia maszyny z innymi procesami oraz dynamicznego zarządzania taktowaniem procesora, co wprowadza zmienność czasu wykonania. Ograniczono ją przez sekwencyjne uruchamianie przebiegów, dziesięciokrotne powtórzenia oraz stosowanie mediany do wyciągania wniosków. Tam, gdzie mimo to pojawiły się wartości odstające, czyli pojedyncze wolniejsze przebiegi wskutek przejściowego ograniczenia taktowania, fakt ten odnotowano jawnie i potraktowano jako artefakt pomiaru, a nie własność konfiguracji.

Nierówny podział dziedziny pojawia się, gdy liczba locale nie dzieli równo boku kostki 1000^3 . Powoduje to nierównomierne obciążenie i zaburza gładkość krzywej skalowania. Nie jest to błąd pomiaru, lecz rzeczywista własność dekompozycji. Uwzględniono ją w interpretacji niemonotonicznego przebiegu skalowania po węzłach.

Trafność zewnętrzna wyników jest ograniczona, jako że uzyskano je na konkretnym, niewielkim klastrze z siecią 1 Gb/s, co oznacza, że wartości bezwzględne nie przenoszą się wprost na inne środowiska. Wnioski mają jednak charakter ogólny i wynikają z właściwości samych obliczeń stencilowych, albowiem obliczenia te są ograniczone przepustowością pamięci oraz komunikacją sieciową, a większe zadania lepiej wykorzystują klaster [9, 1].

Rozdział 4

Wyniki i analiza

4.1 Poprawność i wydajność

4.1.1 Weryfikacja poprawności numerycznej

Przed przystąpieniem do jakichkolwiek pomiarów wydajnościowych konieczne było ustanowienie wiarygodnego kryterium poprawności obliczeń. Jako takie kryterium przyjęto zakres wartości końcowego pola temperatury oraz sumę wszystkich jego komórek.

Warunki początkowe i brzegowe zadania dobrano tak, aby rozwiązanie pozostawało przez cały czas symulacji ograniczone do przedziału $[1, 2]$. Program po zakończeniu obliczeń raportuje statystyki końcowego pola w postaci wpisu `final field: min=1.0 max=2.0 sum=<S>`. Utrzymanie dolnej granicy 1,0 oraz górnej granicy 2,0 jest bezpośrednią konsekwencją zasady maksimum dla równania przewodnictwa cieplnego, ponieważ schemat jawny z odpowiednio dobranym krokiem czasowym nie może wygenerować wartości spoza zakresu wyznaczonego przez dane początkowe i brzegowe. Naruszenie któregośkolwiek z tych ograniczeń natychmiast sygnalizowałoby błąd implementacji schematu różnicowego lub niestabilność numeryczną.

Kluczową rolę w weryfikacji odgrywa jednak suma wszystkich elementów pola, w przybliżeniu równa liczbie komórek siatki, ponieważ poza cienką warstwą gorącą pole pozostaje bliskie wartości 1,0, a sama warstwa gorąca podnosi sumę jedynie nieznacznie. Aby suma nie maskowała lokalnie znoszących się błędów, kontrolę uzupełniają granice $\min = 1,0$ i $\max = 2,0$, potwierdzające utrzymanie zakresu pola, oraz identyczność sumy dla wszystkich podziałów dziedziny, od jednego do dziewięciu locale.

Właśnie na tej podstawie zweryfikowano poprawność obliczeń wielowęzłowych. Przebieg uruchomiony na pojedynczym locale (`-nl 1`) daje pewną wartość sumy

odniesienia. Ten sam problem rozdzielony na N locale daje identyczną sumę *pod warunkiem*, że po każdym kroku czasowym wykonywana jest wymiana warstw brzegowych (halo) za pomocą operacji `updateFluff`. Pomińcie tej wymiany skraca czas wykonania, lecz suma S zaczyna się rozchodzić względem wartości odniesienia, a wynik staje się *cicho błędny*. W przeprowadzonych pomiarach suma końcowa pola wynosiła $1,00518 \cdot 10^9$ i była identyczna dla każdej liczby locale od 1 do 9, co empirycznie potwierdza niezależność wyniku od dekompozycji dziedziny. Wymiana warstw brzegowych jest więc niezbędna do poprawności i nie da się jej pominąć dla zysku wydajności. Suma jest przy tym miarą odporną na błędy systematyczne, do jakich należy pominięcie wymiany halo, które rozchodzi pole w jednym kierunku, a nie na błędy kompensujące się, na przykład zawyżenie wartości w jednej komórce i zniżenie o tyle samo w innej, które nie zmieniają sumy. Zasada maksimum ogranicza każdą komórkę do przedziału $[1, 2]$, przez co punktowe naruszenie tej granicy zostałoby wychwycone niezależnie od sumy. Ta obowiązkowa komunikacja jest głównym wąskim gardłem systemu, co pokazano w podrozdziale 4.2.

4.1.2 Skalowanie wątkowe na pojedynczym węźle

W tej części przeanalizowano wydajność jednowęzłową, czego celem było zbadać, jak czas wykonania programu zależy od liczby wątków obliczeniowych przy ustalonym rozmiarze zadania, a zatem ile faktycznie zyskuje się na zrównolegleniu wewnątrzwęzłowym. Pomiary wykonano dla dziedziny 1000^3 .

Tabela 4.1 przedstawia wyniki tego eksperymentu. Faktyczna liczba zadań qthreads jest tu równa liczbie żądanej.

Tabela 4.1: Skalowanie wątkowe na pojedynczym węźle, dziedzina 1000^3 , `-nl 1, 10` kroków $\times 10$ powtórzeń, Chapel 2.9, kanał MPI, węzeł Pionier.

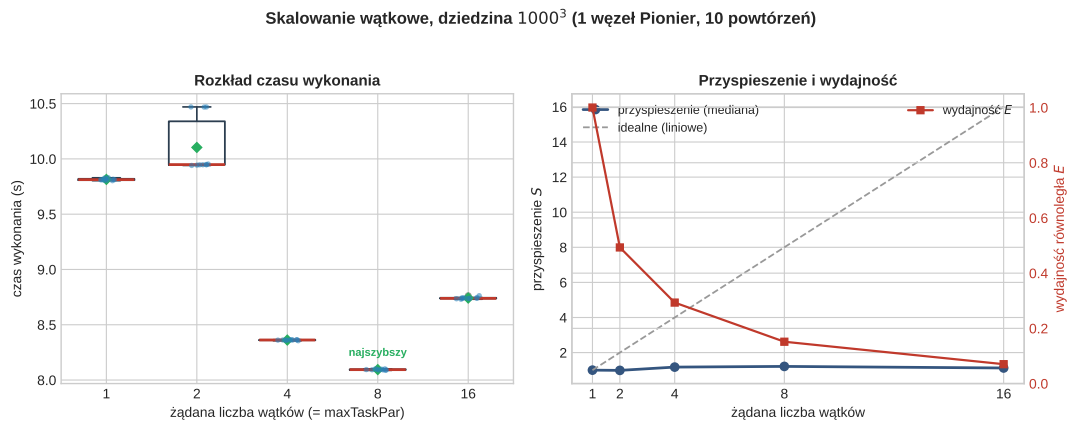
Żądane wątki	<code>maxTaskPar</code>	Mediana [s]	Średnia [s]	Min. [s]	Maks. [s]	Odch. std. [s]	S	E
1	1	9,812	9,814	9,803	9,829	0,008	1,000	1,000
2	2	9,948	10,104	9,941	10,470	0,240	0,986	0,493
4	4	8,362	8,362	8,356	8,365	0,003	1,173	0,293
8	8	8,094	8,095	8,087	8,101	0,004	1,212	0,152
16	16	8,739	8,741	8,731	8,763	0,008	1,123	0,070

Wyniki pokazują, że równoległość wątkowa na pojedynczym węźle przynosi jedynie skromny zysk mimo szesnastu dostępnych wątków. Przejście z jednego do dwóch wątków nie przynosi niemal zysku, a mediana rośnie z 9,812 s do 9,948 s, przy czym trzy z dziesięciu przebiegów dwuwątkowych były wyraźnie wolniejsze, około 10,47 s, wskutek przejściowego ograniczenia taktowania. Dopiero cztery wątki dają

wyraźny skok wydajności, przy którym mediana spada do 8,362s (przyspieszenie 1,173), a optimum osiągane jest przy ośmiu wątkach (8,094s, przyspieszenie 1,212). Dalsze zwiększanie liczby wątków ponownie szkodzi, bo przy szesnastu czas rośnie do 8,739s, toteż maksymalne uzyskane przyspieszenie wynosi tylko około $1,21 \times$.

Wydajność E spada przy tym bardzo ostro, z 1,000 przez 0,493 i 0,293 aż do 0,070 przy szesnastu wątkach. Tak gwałtowny spadek to typowy objaw problemu ograniczonego pasmem pamięci [1]. Schemat różnicowy jest operacją o niskiej intensywności arytmetycznej, dlatego jego przepustowość ogranicza pasmo pamięci głównej, a nie liczba dostępnych rdzeni. Po nasyceniu tego pasma dodatkowe wątki rywalizują o tę samą magistralę, co prowadzi do degradacji wydajności.

Zwiększanie liczby wątków ponad pewien próg nie pomaga tu, a wręcz szkodzi, jako że po nasyceniu magistrali pamięci dodatkowe wątki nie znajdują użytecznej pracy. Rysunek 4.1 ilustruje te zależności, lewy panel przedstawia rozkład czasów wykonania w postaci wykresu pudełkowego, a prawy zestawia zmierzone przyspieszenie z przyspieszeniem idealnym oraz z krzywą wydajności, uwidaczniając odejście od skalowania liniowego już przy dwóch wątkach.



Rysunek 4.1: Skalowanie wątkowe dla dziedziny 1000^3 na węźle Pionier. Lewy panel: rozkład czasu wykonania w postaci wykresu pudełkowego w funkcji liczby wątków. Prawy panel: zmierzone przyspieszenie na tle przyspieszenia idealnego oraz wydajność E .

W przypadku jednowęzłowym równoległość wątkowa przynosi w najlepszym przypadku około 20%-owy zysk, a wydajność szybko załamuje się wraz ze wzrostem liczby wątków. Bezpośrednią przyczyną jest ograniczenie pasmem pamięci, a sytuacja ta pogłębia się dopiero przy skalowaniu na wiele węzłów.

Większy rozstęp przy dwóch wątkach na rysunku 4.1 bierze się z kilku wolniejszych przebiegów, w których procesor na chwilę obniżył taktowanie. To samo zjawisko opisano w podrozdziale 3.2. Takie przebiegi podnoszą średnią, ale nie zmieniają mediany, dlatego jako wynik przyjęto medianę.

4.2 Skalowanie i ograniczenia komunikacji

W tej części przeanalizowano zachowanie programu przy rozdzieleniu obliczeń na wiele węzłów klastra połączonych siecią Ethernet 1 Gb/s. Eksperymenty w tej sekcji przeprowadzono na klastrze dla dziedziny 1000^3 i 100 kroków czasowych. Wykazano, że badany system jest *ograniczony komunikacją*, gdyż to przepustowość sieci, a nie moc obliczeniowa węzłów, wyznacza pułap osiąganey wydajności.

4.2.1 Skalowanie liczbą węzłów

Tabela 4.2 przedstawia czas wykonania w funkcji liczby węzłów, od jednego do dwięciu, dla dziedziny 1000^3 . Oprócz statystyk czasu podano przyspieszenie, wydajność oraz objętość danych wymienianych w ramach warstw brzegowych na jeden krok czasowy w kolumnie „Bajty/krok”.

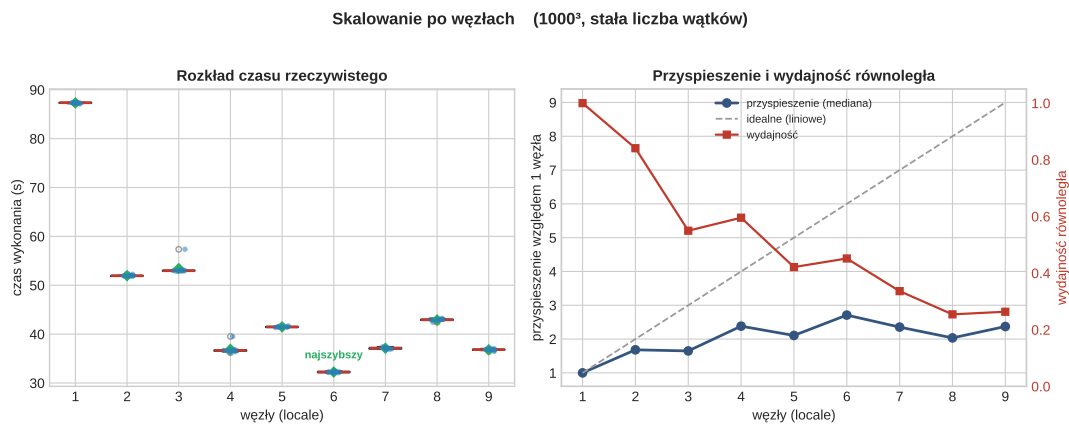
Tabela 4.2: Skalowanie liczbą węzłów, dziedzina 1000^3 , 100 kroków.

Węzły	Mediana [s]	Średnia [s]	Min. [s]	Odch. std. [s]	S	E	Bajty/krok
1	87,344	87,307	87,104	0,092	1,000	1,000	0
2	51,943	51,957	51,858	0,107	1,682	0,841	16000000
3	52,978	53,413	52,862	1,316	1,649	0,550	32000000
4	36,670	36,889	36,203	0,894	2,382	0,595	32032000
5	41,476	41,482	41,347	0,093	2,106	0,421	64000000
6	32,240	32,243	32,076	0,115	2,709	0,452	48064000
7	37,114	37,112	36,909	0,156	2,353	0,336	96000000
8	42,941	42,909	42,419	0,198	2,034	0,254	48096064
9	36,832	36,817	36,469	0,157	2,371	0,263	64128000

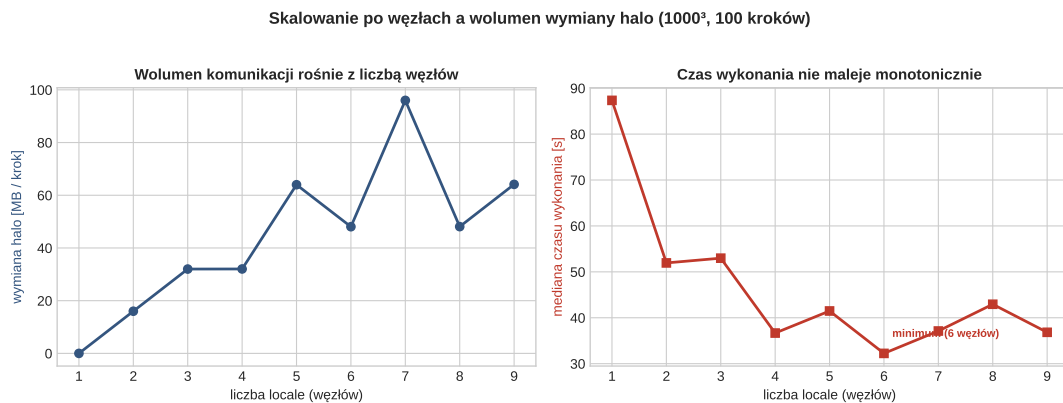
W przypadku skalowania po ilości węzłów zamiast oczekiwanego, systematycznego spadku czasu wraz ze wzrostem liczby węzłów, obserwuje się regularne wahania czasu wykonania pomiędzy parzystymi a nieparzystymi liczbami węzłów. Najlepszy wynik uzyskano dla sześciu węzłów (mediana 32,240 s, przyspieszenie 2,709), podczas gdy sąsiednie konfiguracje pięcio- i siedmiowęzłowa wypadają zauważalnie gorzej (41,476 s oraz 37,114 s). Źródłem tego zjawiska jest nierównomierny podział dziedziny 1000^3 . Gdy liczba węzłów nie dzieli gładko boku kostki, powstaje niezrównoważenie obciążenia, w którym niektóre węzły otrzymują większe poddziedziny i stają się elementem spowalniającym całość, co dodatkowo przekłada się na wzrost danych przesyłanych między węzłami.

Niezależnie od tych oscylacji, przyspieszenie utrzymuje się na poziomie około $2,4\times$ i nie zbliża się do skalowania liniowego, mimo dziewięciokrotnego zwiększenia

liczby węzłów [10]. Wydajność spada z 1,000 do zaledwie 0,263 przy dziewięciu węzłach. Dzieje się tak, ponieważ objętość wymienianych warstw brzegowych rośnie w przybliżeniu liniowo wraz z liczbą węzłów, od 0 bajtów dla pojedynczego węzła, w którym nie ma komunikacji międzywęzłowej, do ponad 64000000 bajtów na krok przy dziewięciu węzłach. Każdy dodany węzeł zwiększa dostępną moc obliczeniową, lecz zarazem powiększa całkowity ruch sieciowy, a przy wolnym łączu narzut komunikacyjny szybko przewyższa zysk obliczeniowy. System jest zatem ograniczony komunikacją.



Rysunek 4.2: Skalowanie liczbą węzłów dla dziedziny 1000³. Lewy panel: rozkład czasu wykonania w funkcji liczby węzłów. Prawy panel: przyspieszenie na tle przyspieszenia idealnego oraz wydajność E .



Rysunek 4.3: Objętość warstw brzegowych wymienianych na krok w megabajtach oraz mediana czasu wykonania w funkcji liczby węzłów.

Zależności te zilustrowano na rysunkach 4.2 oraz 4.3. Pierwszy z nich uwidacznia zarówno oscylacyjny charakter czasu wykonania, jak i znaczne odejście krzywej przyspieszenia od ideału liniowego. Drugi zestawia rosnącą objętość komunikacji z czasem wykonania, czyniąc bezpośrednim widocznym związek między nasilającym się ruchem sieciowym a stagnacją wydajności.

4.2.2 Skalowanie wątkowe przy dziewięciu węzłach

Aby rozstrzygnąć, czy wąskim gardłem w konfiguracji wielowęzłowej jest komunikacja, czy może niedostateczne wykorzystanie rdzeni w obrębie węzła, powtórzono skalowanie wątkowe przy ustalonej liczbie dziewięciu węzłów. Wyniki zawiera tabela 4.3.

Tabela 4.3: Skalowanie wątkowe przy dziewięciu węzłach, dziedzina 1000^3 , 100 kroków.

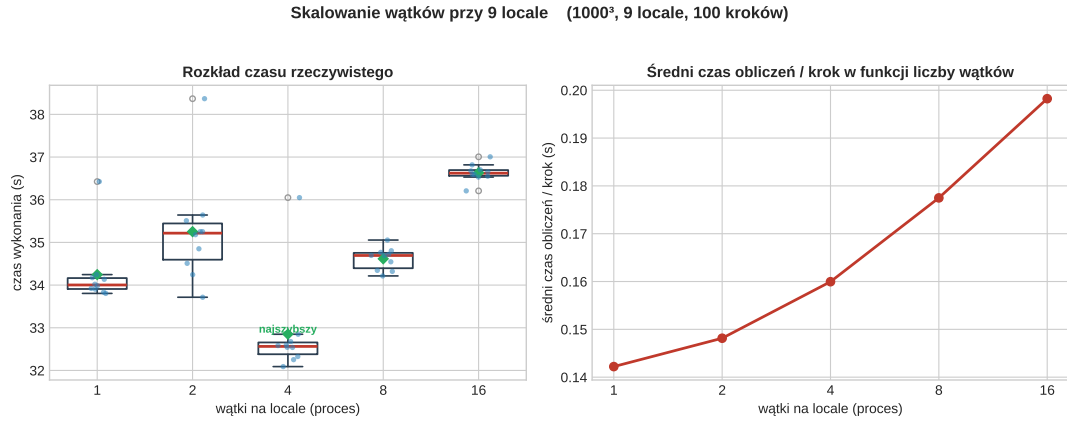
Konfiguracja	Mediana [s]	Średnia [s]	Min. [s]	Odch. std. [s]	S	Obl./krok [s]
1 wątek	34,003	34,245	33,805	0,740	1,000	0,1422
2 wątki	35,217	35,252	33,715	1,184	0,966	0,1481
4 wątki	32,563	32,850	32,090	1,086	1,044	0,1600
8 wątków	34,694	34,616	34,215	0,245	0,980	0,1775
16 wątków	36,623	36,633	36,208	0,195	0,928	0,1982

Odpowiedź jest jednoznaczna, gdyż czas wykonania pozostaje niemal płaski w zakresie od około 32 do 37 sekund niezależnie od liczby wątków, z płytkim minimum przy czterech wątkach (32,563 s). Przyspieszenie oscyluje wokół jedności i nigdy nie przekracza 1,044. Najbardziej wymowna jest jednak kolumna średniego czasu obliczeń na krok, który rośnie *monotonicznie* wraz z liczbą wątków, od 0,1422 s przy jednym wątku do 0,1982 s przy szesnastu. Gdyby problem był ograniczony mocą obliczeniową, dodawanie wątków skracałoby czas obliczeń na krok, tymczasem obserwuje się zjawisko przeciwne, czyli jego wzrost. Nie jest to skutek nadmiernej liczby wątków względem rdzeni, bo ich liczba nie przekracza szesnastu dostępnych wątków wydajnościowych ($p \leq 16$), lecz rywalizacji o zasoby współdzielone, w której dodatkowe wątki nie znajdują użytecznej pracy, a jedynie konkurują o pasmo pamięci oraz z wątkiem odpytującym warstwę komunikacyjną GASNet [16]. Zwiększanie liczby wątków nie pomaga, skoro problem jest ograniczony komunikacją, a nie obliczeniami. Rysunek 4.4 przedstawia te zależności graficznie.

Przy dziewięciu węzłach pojedyncze przebiegi bardziej się od siebie różnią, najbardziej przy dwóch wątkach. Wynika to z samej komunikacji, bo wszystkie węzły dzielą jedno łącze 1 Gb/s i jego obciążenie bywa różne w kolejnych przebiegach. Liczba wątków nie ma tu większego znaczenia, stąd mediana pozostaje zbliżona.

4.2.3 Skalowanie rozmiaru zadania

Ostatni zestaw eksperymentów bada wpływ rozmiaru dziedziny na wydajność przy ustalonej liczbie dziewięciu węzłów. Tabela 4.4 zestawia czasy wykonania, przepustowość obliczeniową wyrażoną w miliardach komórek na sekundę (mld komórek/s)



Rysunek 4.4: Skalowanie wątkowe przy dziewięciu węzłach, dziedzina 1000³. Lewy panel: rozkład czasu wykonania. Prawy panel: średni czas obliczeń na krok w funkcji liczby wątków, rosnący wskutek rywalizacji o zasoby współdzielone.

oraz udział komunikacji `comm_frac` (3.3).

Tabela 4.4: Skalowanie rozmiaru dziedziny przy dziewięciu węzłach, 100 kroków.

Rozmiar	Mediana [s]	Średnia [s]	Przepustowość [mld komórek/s]	<code>comm_frac</code>
125 ³	1,668	1,670	0,117	0,963
250 ³	2,640	2,652	0,592	0,947
500 ³	7,625	7,673	1,639	0,729
1000 ³	36,741	36,802	2,722	0,454

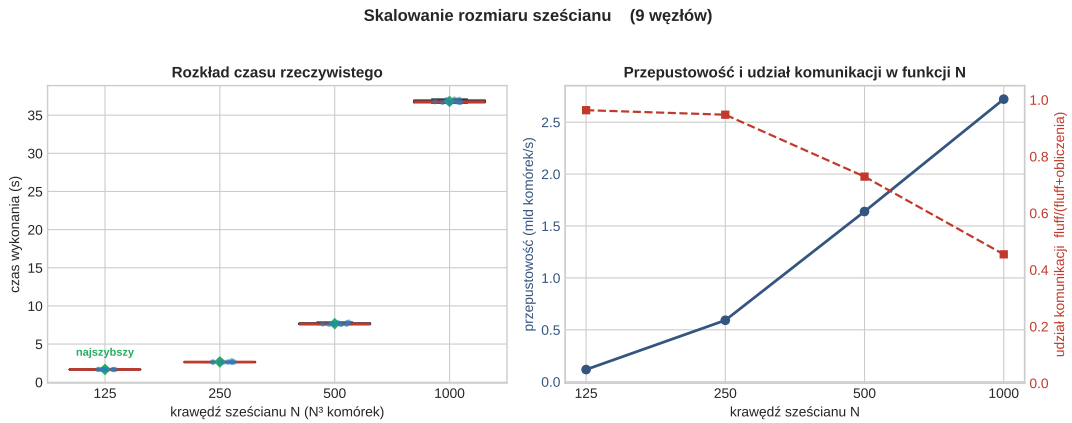
Wynik ten najlepiej potwierdza, że ograniczenie ma charakter komunikacyjny. Przepustowość rośnie systematycznie wraz z rozmiarem dziedziny, od zaledwie 0,117 mld komórek/s dla najmniejszej kostki 125³ do 2,722 mld komórek/s dla największej kostki 1000³. Jednocześnie udział komunikacji `comm_frac` maleje monotonicznie z 0,963 do 0,454. Innymi słowy, dla małych dziedzin blisko 96 % czasu pochłania komunikacja, a faktyczne obliczenia stanowią margines. Wraz ze wzrostem dziedziny proporcje te wyrównują się i przy 1000³ komunikacja oraz obliczenia zajmują już zbliżony czas. Nawet przy największej zbadanej kostce komunikacja wciąż odpowiada za blisko połowę czasu kroku. Trend jest wprawdzie korzystny, lecz w osiągalnym zakresie rozmiarów zadanie nie przestaje być istotnie obciążone komunikacją.

Mechanizm tego zjawiska jest geometryczny, ponieważ objętość dziedziny rośnie jak N^3 , a powierzchnia warstw brzegowych jak N^2 , więc wraz ze wzrostem N poprawia się stosunek pracy obliczeniowej do komunikacji. Duże kostki znacznie lepiej wykorzystują klaster niż małe, zdominowane przez narzut komunikacyjny.

Rozmiar wymiany halo zależy od geometrycznego układu dekompozycji. Dla zilustrowania rozważmy dwuwymiarową dziedzinę 100×100 podzieloną na cztery

locale. Przy podziale plastrami wzdłuż jednej osi każdy blok ma wymiary 25×100 , dwa skrajne locale wysyłają po 100 komórek brzegowych, a dwa wewnętrzne po 200, co daje łącznie 600 komórek wymienianych na krok. Przy podziale na bloki kwadratowe 50×50 każdy z czterech locale wysyła do dwóch sąsiadów brzeg poziomy i pionowy po 50 komórek, co daje $4 \cdot (50 + 50) = 400$ komórek. W obu przypadkach objętość wymiany jest rzędu pola powierzchni granicy, a nie objętości, co w trzech wymiarach przekłada się na omówioną zależność N^2 .

Największą zbadaną dziedziną jest 1000^3 . Omówione zależności ilustruje rysunek 4.5.



Rysunek 4.5: Skalowanie rozmiaru dziedziny przy dziewięciu węzłach. Lewy panel: rozkład czasu wykonania dla kolejnych rozmiarów. Prawy panel: przepustowość oraz udział komunikacji `comm_frac` w funkcji rozmiaru kostki.

4.2.4 Synteza: system ograniczony komunikacją

Wyniki z tego rozdziału wskazują wspólnie, że system jest ograniczony komunikacją i przepustowością pamięci, a nie mocą obliczeniową. Przyspieszenie węzłowe utrzymuje się na poziomie około $2,4\times$ (tabela 4.2), daleko od skalowania liniowego, mimo dziewięciokrotnego zwiększenia liczby węzłów. Zwiększanie liczby wątków na węzeł nie przynosi poprawy, a w konfiguracji wielowęzłowej średni czas obliczeń na krok wręcz rośnie z liczbą wątków (tabela 4.3), co wskazuje na nasycenie zasobów współdzielonych, a nie na niedobór mocy obliczeniowej. Udział komunikacji dominuje przy małych dziedzinach i maleje dopiero dla dużych kostek (tabela 4.4), zgodnie z geometryczną relacją między objętością N^3 a powierzchnią N^2 . Na każdym węźle jeden wątek warstwy komunikacyjnej GASNet obciąża rdzeń niemal w całości, prowadząc aktywne odpytywanie (ang. *busy-polling*) w oczekiwaniu na postęp komunikacji, co potwierdzono analizą stosów wywołań w debuggerze. To nie jest obciążenie obliczeniowe, lecz widoczny i mierzalny skutek ograniczenia komunikacją.

Fizycznym ograniczeniem jest łącze Ethernet o przepustowości 1 Gb/s, które dla dziedziny rzędu 1000^3 ulega nasyceniu. Objętość warstw brzegowych na krok rośnie zarówno z liczbą węzłów, jak i z kwadratem boku dziedziny, wobec czego to sieć, a nie procesory, wyznacza pułap wydajności.

Osobnego omówienia wymaga dobór kanału komunikacyjnego GASNet, który okazał się decydujący dla tego, czy obliczenia w ogóle kończą się poprawnie [15]. W pomiarach kanał `udp` przerywał działanie błędem `ECONGESTION` pod obciążeniem typu incast, natomiast kanał `mpi` kończył obliczenia poprawnie, choć wolniej. Z tego powodu nowsza seria pomiarów wykorzystuje kanał MPI, ponieważ odporność na zawodną sieć jest tu ważniejsza od szybkości.

Skoro system jest ograniczony komunikacją, dalszej poprawy wydajności nie należy szukać ani w zwiększaniu liczby wątków, ani w dokładaniu kolejnych węzłów, lecz przede wszystkim w przyspieszeniu warstwy komunikacyjnej, czy to przez zastosowanie szybszego łącza sieciowego, czy przez wybór wydajniejszego kanału komunikacyjnego GASNet. Dopóki halo wymieniane jest przez łącze 1 Gb/s, klastery wciąż czeka na dane z sieci, a część rdzeni pozostaje niewykorzystana.

4.2.5 Zestawienie zbiorcze i powtarzalność

Dla wygody czytelnika zebrano najważniejsze wnioski ilościowe z poszczególnych grup eksperymentów w jednym miejscu. Tabela 4.5 podaje dla każdej serii kluczową miarę oraz jej najistotniejszy wynik.

Tabela 4.5: Zestawienie zbiorcze najważniejszych wyników eksperymentalnych rozdziału.

Grupa eksperymentów	Kluczowa miara	Wynik
Wątki 1000^3 (tabela 4.1)	optimum / maks. przyspieszenie	8 wątków, $1,21\times$
Węzły 1000^3 (tabela 4.2)	optimum / plateau przyspieszenia	6 węzłów, około $2,4\times$
Wątki, 9 węzłów (tabela 4.3)	przyspieszenie (płaskie)	$\leq 1,044$
Rozmiar kostki (tabela 4.4)	przepustowość	0,117–2,722 mld komórek/s
Rozmiar kostki (tabela 4.4)	udział komunikacji <code>comm_frac</code>	$0,963 \rightarrow 0,454$

Z zestawienia widać, że niezależnie od osi skalowania, liczby wątków, liczby węzłów czy rozmiaru kostki, wyniki trafiają na to samo ograniczenie, czyli pasmo pamięci i komunikację. Jediną osią, wzdłuż której obserwuje się wyraźną poprawę efektywności wykorzystania klastra, jest rozmiar zadania, gdzie powiększanie kostki

przesuwa proporcje na korzyść obliczeń, zgodnie z omówioną wcześniej relacją N^3 do N^2 .

Odrębnego komentarza wymaga powtarzalność pomiarów, stanowiąca podstawę zaufania do prezentowanych median. Zdecydowana większość serii charakteryzuje się bardzo niskim rozrzutem wyników. Dla serii 1000^3 na węźle Pionier (tabela 4.1) odchylenie standardowe czasu wykonania nie przekracza 0,07 s, a dla większości konfiguracji jest rzędu zaledwie kilku tysięcznych sekundy. Tak niski rozrzut oznacza, że pojedynczy przebieg jest w tych przypadkach niemal równie miarodajny co mediana z dziesięciu powtórzeń, a różnice między konfiguracjami to rzędy dziesiątych sekundy lub więcej, więc nie da się ich wytłumaczyć samym szumem pomiarowym.

Osobną kwestią jest praktyczna granica rozmiaru zadania. Dziedzin istotnie większych niż 1000^3 nie udało się ukończyć, gdyż przekraczają one możliwości pamięciowe i czasowe klastra, wobec czego nie figurują w żadnej tabeli i wyznaczają jedynie górny kres rozmiaru osiągalnego na badanej infrastrukturze. Świadomość tego ograniczenia nie podważa głównych wniosków rozdziału, które opierają się na seriach o niskim rozrzucie i dziesięciokrotnej powtarzalności, lecz trzeba o niej pamiętać, oceniając, do jakich rozmiarów zadania sięgają wnioski tej pracy.

Rozdział 5

Podsumowanie i wnioski

5.1 Wnioski

Celem pracy było zaprojektowanie, zaimplementowanie i wdrożenie na klastrze rozproszonego programu symulacyjnego rozwiązującego trójwymiarowe równanie przewodnictwa ciepła w języku Chapel oraz przeprowadzenie systematycznej analizy jego poprawności i skalowalności. Zrealizowano wszystkie założone cele, a wyniki potwierdzają tezę pracy. Dla rozważanej klasy obliczeń stencilowych czynnikiem ograniczającym wydajność jest nie moc obliczeniowa, lecz przepustowość pamięci na poziomie węzła oraz koszt komunikacji między węzłami [9].

W zakresie poprawności wykazano, że rozwiązanie różnicowe daje wynik niezależny od liczby locale, na które rozdzielono dziedzinę, pod warunkiem prawidłowej wymiany warstw brzegowych. Niezmiennik sumy pola w stanie końcowym okazał się prostym, lecz skutecznym kryterium kontrolnym. Pominięcie wymiany halo znacząco skraca czas wykonania, lecz prowadzi do wyniku *cicho błędnego*, co pokazuje, że w programowaniu rozproszonym krótki czas wykonania nie oznacza jeszcze poprawnego wyniku.

Zrównoleglenie wątkowe w obrębie węzła nasycza się bardzo szybko. Dla kostki 1000^3 optimum przypada na osiem wątków, a maksymalne przyspieszenie nie przekracza około $1,21\times$ mimo szesnastu dostępnych wątków, co jest sygnaturą ograniczenia przepustowością pamięci. Skalowanie po węzłach zatrzymuje się na poziomie około $2,4\times$ i pozostaje daleko od zależności liniowej, a jego przebieg jest niemonotoniczny, z najlepszym czasem przy sześciu węzłach, wskutek nierównego podziału kostki 1000^3 dla „niewygodnych” dzielników oraz rosnącej objętości wymiany halo. Przy łączu 1 Gb/s czyni to obliczenia ograniczonymi komunikacją. Poprawę przynosi dopiero powiększanie zadania. Wraz ze wzrostem krawędzi kostki przepustowość rośnie z 0,117 do 2,722 mld komórek na sekundę, a udział komunikacji maleje z 0,96 do 0,45, ponieważ objętość rośnie z sześcianem krawędzi, a powierzchnia wymiany

halo jedynie z jej kwadratem.

Praktyczne znaczenie miał dobór kanału komunikacyjnego GASNet. Wariant AMUDP, choć prosty, ulega awarii typu *ECONGESTION* pod obciążeniem typu *in-cast* i przy stratach pakietów, bo nie zapewnia kontroli przepływu [19], natomiast mpi-conduit oparty na bibliotece MPICH i protokole TCP jest na nie odporne dzięki kontroli przepływu i retransmisji, kosztem nieco niższej wydajności. Wyższa warstwa języka również miała znaczenie. Globalny widok tablic, gotowy rozkład *StencilDist* z automatyczną wymianą otoczek oraz zwięzłe iteratory równoległe pozwoliły zapisać poprawny kod rozproszony znacznie prościej niż przy jawnym użyciu MPI [4, 11]. Abstrakcja ta nie zniosła jednak fundamentalnego ograniczenia komunikacyjnego. Koszt wymiany halo pozostaje widoczny w wynikach i wyznacza granice skalowania. Model programowania nie zmienia właściwości sieci, co w tym przypadku oznacza, że produktywność i wydajność się uzupełniają, a nie zastępują.

5.2 Ograniczenia

Uzyskane wyniki należy interpretować z uwzględnieniem kilku ograniczeń.

Środowiskiem docelowym był niewielki, jednorodny klaster połączony siecią Ethernet o przepustowości 1 Gb/s, która jest wąskim gardłem i w znacznej mierze determinuje granice skalowania po węzłach. Na infrastrukturze z szybszą siecią o małych opóźnieniach, na przykład InfiniBand, można by oczekiwać lepszego skalowania oraz przesunięcia punktu, w którym program przestaje być ograniczony komunikacją, choć weryfikacja tej hipotezy wymagałaby odpowiedniego sprzętu. Praktyczną granicą jest też maksymalny rozmiar zadania. Dziedzin istotnie większych niż 1000^3 nie udało się zmieścić w pamięci i czasie klastra.

Zakres pracy świadomie zawężono do jawnej metody różnic skończonych na siatce regularnej z arytmetyką podwójnej precyzji. Metody niejawne, które są bezwarunkowo stabilne, lecz wymagają rozwiązywania układów równań, siatki niestrukturalne, techniki nakładania komunikacji na obliczenia oraz akceleracja na procesorach graficznych wykraczają poza ramy niniejszej pracy, a każde z nich mogłoby zmienić charakterystykę obliczeniową problemu. Wreszcie badania dotyczyły wyłącznie skalowania silnego, przy ustalonym rozmiarze zadania i rosnącej liczbie jednostek równoległości. Skalowanie słabe, gdzie rozmiar zadania rośnie proporcjonalnie do liczby locale, dałoby obraz uzupełniający.

5.2.1 Kierunki dalszych prac

Te ograniczenia wyznaczają naturalne kierunki dalszych prac.

Najbardziej obiecującym z nich jest nakładanie komunikacji na obliczenia. Wy-
mianę warstw brzegowych można rozpocząć asynchronicznie i w tym czasie liczyć
wnętrze poddziedziny, które nie zależy od danych sąsiadów, ujawniając komunika-
cję dopiero przy aktualizacji komórek przylegających do granicy. W języku Chapel
wyraża się to naturalnie, za pomocą zadań asynchronicznych i jawnego rozdzielania
wnętrza od brzegu poddziedziny.

Powtórzenie pomiarów na sieci o niskim opóźnieniu i wysokiej przepustowości,
na przykład InfiniBand, wraz z kanałem komunikacyjnym GASNet dobranym do ta-
kiej sieci pozwoliłoby ustalić, w jakim stopniu obserwowane granice skalowania wy-
nikają z samego algorytmu, a w jakim z ograniczeń konkretnego łącza 1 Gb/s. Uzu-
pełniającą warto rozważyć skalowanie słabe, gdzie rozmiar zadania rośnie proporcjo-
nalnie do liczby locale, co pozwoliłoby oddzielić koszty stałe od kosztów rosnących
wraz z rozmiarem problemu.

Innym naturalnym rozszerzeniem są warianty numeryczne i sprzętowe, czyli sche-
maty niejawne, które są bezwarunkowo stabilne, lecz wymagają rozwiązywania ukła-
dów równań i globalnej komunikacji innego typu, siatki niestrukturalne oraz akcele-
racja jądra obliczeniowego na procesorach graficznych, którą Chapel wspiera odpo-
wiednimi locale i atrybutami pętli równoległych. Każde z nich zmieniałoby charak-
terystkę obliczeniową problemu i pozwoliłoby sprawdzić, na ile wnioski tej pracy
zachowują ważność poza klasą jawnych schematów stencilowych.

Z kolei inżynierskim uzupełnieniem byłoby pełne zautomatyzowanie potoku po-
miarowego, od budowy i dystrybucji po zbieranie logów i generowanie wykresów, tak
aby cały cykl badawczy był odtwarzalny jednym poleceniem. Skrypt `bench.sh` au-
tomatyzuje już same serie pomiarowe, a do pełnego potoku brakuje spięcia budowy,
dystrybucji i rysowania wykresów w jedno polecenie. Wprowadzone w tej pracy
elementy, czyli samoopisujące się logi oraz skrypty budowy, dystrybucji, pomiaru
i analizy, mogą być bazą takiego potoku.

Bibliografia

- [1] K. Datta i in., *Stencil Computation Optimization and Auto-tuning on State-of-the-Art Multicore Architectures*, Proceedings of the ACM/IEEE Conference on Supercomputing (SC'08), 2008.
- [2] Message Passing Interface Forum, *MPI: A Message-Passing Interface Standard, Version 4.1*, 2023.
- [3] G. Almasi, *PGAS (Partitioned Global Address Space) Languages*, w: *Encyclopedia of Parallel Computing*, Springer, 2011, s. 1539–1545.
- [4] B. L. Chamberlain, D. Callahan, H. P. Zima, *Parallel Programmability and the Chapel Language*, International Journal of High Performance Computing Applications, 21(3), 2007, s. 291–312.
- [5] S. Falkowski, S. Orzechowski, S. Wojciechowski, M. Nowicki, *Parallel Thermal Conductivity Simulation on a Raspberry Pi Cluster: A Comparative Analysis of Chapel, C#, and Julia*, w: *Proceedings of the 16th International Conference on Parallel Processing and Applied Mathematics (PPAM 2026)*, Lecture Notes in Computer Science, Springer, 2026 (w druku).
- [6] R. J. LeVeque, *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*, SIAM, Philadelphia, 2007.
- [7] J. C. Strikwerda, *Finite Difference Schemes and Partial Differential Equations*, wyd. 2, SIAM, Philadelphia, 2004.
- [8] K. W. Morton, D. F. Mayers, *Numerical Solution of Partial Differential Equations*, wyd. 2, Cambridge University Press, 2005.
- [9] S. Williams, A. Waterman, D. Patterson, *Roofline: An Insightful Visual Performance Model for Multicore Architectures*, Communications of the ACM, 52(4), 2009, s. 65–76.

- [10] G. M. Amdahl, *Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities*, AFIPS Conference Proceedings, vol. 30, 1967, s. 483–485.
- [11] B. L. Chamberlain, *Chapel*, w: P. Balaji (red.), *Programming Models for Parallel Computing*, MIT Press, 2015, s. 129–159.
- [12] UPC Consortium, *UPC Language Specifications, v1.3*, Lawrence Berkeley National Laboratory, LBNL-6623E, 2013.
- [13] The Chapel Team, *The Stencil Distribution (**StencilDist**): Standard Module Documentation*, 2026. <https://chapel-lang.org/docs/modules/dists/StencilDist.html>.
- [14] K. B. Wheeler, R. C. Murphy, D. Thain, *Qthreads: An API for Programming with Millions of Lightweight Threads*, IEEE International Symposium on Parallel and Distributed Processing (IPDPS), 2008.
- [15] D. Bonachea, P. H. Hargrove, *GASNet-EX: A High-Performance, Portable Communication Library for Exascale*, Languages and Compilers for Parallel Computing (LCPC 2018), LNCS 11882, Springer, 2019.
- [16] D. Bonachea, *GASNet Specification*, Technical Report UCB/CSD-02-1207, University of California, Berkeley, 2002 (rewidowana do wersji 1.8).
- [17] The Chapel Team, *Chapel Language Specification and User Documentation*, wersja 2.9, Chapel a Series of LF Projects, LLC, 2026. <https://chapel-lang.org>.
- [18] W. Gropp, *MPICH2: A New Start for MPI Implementations*, Recent Advances in PVM and MPI, LNCS 2474, Springer, 2002.
- [19] Y. Chen, R. Griffith, J. Liu, R. H. Katz, A. D. Joseph, *Understanding TCP Incast Throughput Collapse in Datacenter Networks*, Proceedings of the 1st ACM Workshop on Research on Enterprise Networking (WREN), 2009.
- [20] P. Luszczek, J. J. Dongarra, i in., *Introduction to the HPC Challenge Benchmark Suite*, Technical Report ICL-UT-05-01, University of Tennessee, 2005.